ingenieur
wissenschaften
htw saar

Hochschule für
Technik und Wirtschaft
des Saarlandes
University of
Applied Sciences

Bachelor-Thesis

zur Erlangung des akademischen Grades
Bachelor of Science (B. Sc.)
an der Hochschule für Technik und Wirtschaft des Saarlandes
im Studiengang Praktische Informatik
der Fakultät für Ingenieurwissenschaften

Evaluierung lokaler KI-Modelle auf Android für BCI-Anwendungen

vorgelegt von
Ahmed Chaabo

betreut und begutachtet von
Prof. Dr. rer. nat. Dr. rer. med. habil. Daniel J.Strauss
Prof. Dr.-ing. Klaus Berberich

Saarbrücken, Tag. Monat Jahr

Selbständigkeitserklärung

Ich versichere, dass ich die vorliegende Arbeit (bei einer Gruppenarbeit: den entsprechend gekennzeichneten Anteil der Arbeit) selbständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe.
Ich erkläre hiermit weiterhin, dass die vorgelegte Arbeit zuvor weder von mir noch von einer anderen Person an dieser oder einer anderen Hochschule eingereicht wurde.
Darüber hinaus ist mir bekannt, dass die Unrichtigkeit dieser Erklärung eine Benotung der Arbeit mit der Note „nicht ausreichend“ zur Folge hat und einen Ausschluss von der Erbringung weiterer Prüfungsleistungen zur Folge haben kann.

Saarbrücken, Tag. Monat Jahr

Ahmed Chaabo

Zusammenfassung

Kurze Zusammenfassung des Inhaltes in deutscher Sprache, der Umfang beträgt zwischen einer halben und einer ganzen DIN A4-Seite.

Orientieren Sie sich bei der Aufteilung bzw. dem Inhalt Ihrer Zusammenfassung an Kent Becks Artikel: `http://plg.uwaterloo.ca/~migod/research/beckOOPSLA.html`.

If the human brain were so simple that we could understand it,
we would be so simple that we couldn't.

— Emerson M. Pugh

Danksagung

Ich möchte mich bei allen bedanken, die mich während der Bearbeitung dieser Arbeit unterstützt haben.

Zudem gilt mein besonderer Dank meiner Familie und meinen Freunden, die mich während der Zeit im Studium motiviert und mir den Rücken gestärkt haben.

Abschließend bedanke ich mich bei meinen Betreuern, Prof. Dr. rer. nat. Dr. rer. med. habil. Daniel J. Strauss und Tobias Grün M.Sc., für die Unterstützung und die Möglichkeit, diese Arbeit im Rahmen des Projekts UnicornART zu schreiben.

Inhaltsverzeichnis

1 Einleitung **1**
- 1.1 Motivation . . . 1
- 1.2 Zielsetzung . . . 1
- 1.3 Aufbau der Arbeit . . . 2

2 Grundlagen **3**
- 2.1 Elektroenzephalographie (EEG) . . . 3
- 2.2 Frequenzbänder . . . 3
- 2.3 Frequenzanalyse . . . 4
  - 2.3.1 Fourier-Transformation . . . 4
  - 2.3.2 Fensterung und spektrale Leckage . . . 4
  - 2.3.3 Welch-Methode . . . 5
- 2.4 Lokale Sprachmodelle auf mobilen Geräten . . . 5
  - 2.4.1 Large Language Models . . . 5
  - 2.4.2 On-Device-Inferenz und Quantisierung . . . 5
  - 2.4.3 Modell und Laufzeitumgebung . . . 6
  - 2.4.4 Systemintegrierte und frei deploybare Modelle . . . 6
- 2.5 Brain-Computer-Interface (BCI) . . . 7

3 Stand der Technik **9**
- 3.1 On-Device-Inferenz-Lösungen . . . 9
  - 3.1.1 Systemintegrierte Lösung . . . 9
  - 3.1.2 Frei deploybare Lösung . . . 9
- 3.2 Die Gemma-Modellfamilie . . . 10
- 3.3 Die Qwen-Modellfamilie . . . 10
- 3.4 Bestehende Anwendung . . . 10
  - 3.4.1 Unicorn Hybrid Black . . . 11
  - 3.4.2 Datenübertragung und Anbindung . . . 11
  - 3.4.3 Funktionsumfang der Anwendung . . . 12
  - 3.4.4 Beitrag dieser Arbeit . . . 12

4 Methodik **13**
- 4.1 Merkmalsextraktion . . . 13
  - 4.1.1 Segmentierung . . . 13
  - 4.1.2 Spektrale Merkmale . . . 14
  - 4.1.3 Zeitstatistische Merkmale . . . 15
  - 4.1.4 Aufbereitung für das Sprachmodell . . . 15
- 4.2 Modellauswahl . . . 16
- 4.3 Anwendungsszenarien . . . 16
- 4.4 Messprotokoll . . . 16
  - 4.4.1 Metriken . . . 17
  - 4.4.2 Messbedingungen . . . 18

5 Implementierung **19**
5.1 Architektur . . . 19
5.2 Implementierung der Merkmalsextraktion . . . 21
5.3 Integration des Sprachmodells . . . 25
5.4 Prompt-Design und iterative Optimierung . . . 27
5.5 Benutzeroberfläche . . . 28
5.6 Benchmark . . . 28

6 Evaluation **31**
6.1 Durchführung der Messungen . . . 31
6.2 Latenz . . . 31
6.3 Ressourcenverbrauch . . . 32
6.4 Stabilität . . . 32
6.5 Ausgabequalität . . . 33
6.5.1 Bewertungskriterien . . . 33
6.5.2 Beispielausgaben je Szenario . . . 33
6.6 Zusammenfassende Gegenüberstellung . . . 33

7 Diskussion **35**
7.1 Einordnung der Ergebnisse . . . 35
7.2 Einschränkungen . . . 35
7.3 Handlungsempfehlungen . . . 35

8 Fazit und Ausblick **37**
8.1 Fazit . . . 37
8.2 Ausblick . . . 37

Literatur **39**

Abbildungsverzeichnis **41**

Tabellenverzeichnis **41**

Listings **41**

Abkürzungsverzeichnis **43**

A Erster Abschnitt des Anhangs **47**

1 Einleitung

Gehirn-Computer-Schnittstellen und lokal ausführbare künstliche Intelligenz haben sich in den letzten Jahren stark weiterentwickelt. Die Kombination beider Technologien eröffnet neue Möglichkeiten, unter anderem den Einsatz lokaler Sprachmodelle zur Verarbeitung von EEG-Daten auf einem mobilen Android-Gerät. Die vorliegende Arbeit untersucht wie diese Kombination in der Praxis funktioniert. Dieses Kapitel gibt einen Überblick über die Motivation, Zielsetzung und den Aufbau der Arbeit.

1.1 Motivation

Die Elektroenzephalographie (EEG) ist ein etabliertes Verfahren zur nicht-invasiven Aufzeichnung der elektrischen Aktivität des Gehirns [3]. Eine Gehirn-Computer-Schnittstelle (Brain-Computer-Interface, BCI) zielt auf dieser Grundlage darauf ab, kognitive Zustände in Echtzeit zu identifizieren, um eine Steuerung und Kommunikation ohne jeglichen Einsatz von Muskeln zu ermöglichen [3]. Vor allem körperlich stark beeinträchtigte Menschen profitieren von BCI-Systemen. Gelähmte Personen ist es so möglich, BCI-basierte Buchstabiersysteme zu nutzen, die einen wichtigen Kommunikationskanal darstellen [3].

Traditionelle EEG-Laborhardware, wie sie oft stationär verwendet wird, ist groß und unpraktisch für den Gebrauch außerhalb einer Laborumgebung. Demgegenüber erlauben moderne, kabellose und am Kopf getragene EEG-Geräte (wie das Unicorn Hybrid Black) inzwischen auch Messungen außerhalb des Labors [3, 6].

Dennoch ist das Deuten der aufgezeichneten Signale komplex und benötigt oft Fachwissen. Gleichzeitig haben große Sprachmodelle (Large Language Models, LLMs) die Verarbeitung natürlicher Sprache grundlegend revolutioniert [29]. Da lokal ausführbare Inferenzen keine Cloud-Anbindung benötigen und zudem datenschutzfreundlich sind, gibt es ein wachsendes Interesse daran, LLMs direkt auf Endgeräten auszuführen [29].

Durch die Kombination lokaler Sprachmodelle und der aufbereiteten EEG-Daten ist es möglich, verständliche und textuelle Beschreibungen zu erzeugen. Der Betrieb solcher Modelle auf Mobilgeräten wird jedoch durch erhebliche rechentechnische Limitierungen sowie durch einen stark begrenzten Arbeits- und Energiespeicher eingeschränkt [29]. Inwieweit sich lokale Sprachmodelle unter diesen ressourcenbeschränkten Bedingungen tatsächlich für die Verarbeitung von EEG-Daten eignen, ist bislang offen.

1.2 Zielsetzung

Ziel dieser Arbeit ist die systematische Untersuchung, ob und inwieweit sich lokal auf Android-Geräten ausgeführte Sprachmodelle für die Verarbeitung von EEG-Daten im Rahmen mobiler Brain-Computer-Interface-Anwendungen eignen. Damit stellt sich folgende Forschungsfrage:

> *Inwieweit eignen sich lokal auf Android-Geräten ausgeführte Sprachmodelle für die Verarbeitung von EEG-Daten im Rahmen mobiler Brain-Computer-Interface-Anwendungen?*

Um diese Frage zu beantworten, wird eine bestehende Android-EEG-Anwendung um lokale Sprachmodelle erweitert, die dann aufbereitete EEG-Merkmale in verständliche, textuelle Beschreibungen übersetzen. Dabei werden drei verschiedene Anwendungsszenarien sowie drei verschiedene Sprachmodelle mit unterschiedlicher Größe und unterschiedlichem Hersteller betrachtet. So ist es möglich, sowohl den Einfluss der Modellgröße als auch des Herstellers auf die jeweiligen Szenarien zu untersuchen.

Die Eignung wird schlussendlich anhand verschiedener Kriterien bewertet. Dazu zählen Latenz, Ressourcenverbrauch (Arbeitsspeicher und Akku), Stabilität sowie die Qualität der ausgegebenen Texte. Darüber hinaus werden praktische Einschränkungen wie die Geräteabhängigkeit systemintegrierter Modelle dokumentiert. Das Ergebnis dieser Arbeit liefert einen systematischen Vergleich, aus dem sich Handlungsempfehlungen für den Einsatz lokaler Sprachmodelle in mobilen BCI-Anwendungen ableiten lassen.

Der Fokus liegt dabei nicht auf dem Training der Modelle, sondern auf ihrer Integration, Leistungsfähigkeit und ihren Grenzen im mobilen Einsatz.

1.3 Aufbau der Arbeit

Der Aufbau der Arbeit gliedert sich in acht Kapitel.

Kapitel 2 vermittelt alles an notwendigem Wissen, das für das Verständnis der Arbeit erforderlich ist. Kapitel 3 beschreibt den derzeitigen Stand der Technik. Kapitel 4 erläutert die Methodik, also wie die Forschungsfrage beantwortet werden soll. Kapitel 5 stellt dar, wie die konkrete Implementierung der Anwendung erfolgt ist. Kapitel 6 präsentiert die Ergebnisse der Evaluation. Kapitel 7 diskutiert diese Ergebnisse sowie die praktischen Einschränkungen. Kapitel 8 schließt die Arbeit mit einem Fazit und einem Ausblick auf mögliche Weiterentwicklungen ab.

2 Grundlagen

In diesem Kapitel werden die Grundlagen, die für das Verständnis der Arbeit benötigt werden, vermittelt. Dabei werden Folgenden zentrale Konzepte vorgestellt, auf denen die weiteren Kapitel aufbauen.

2.1 Elektroenzephalographie (EEG)

Die Elektroenzephalographie (EEG) ist ein Verfahren zur Aufzeichnung der elektrischen Aktivität des menschlichen Gehirns. Die aufgezeichneten Signale bestehen aus Spannungsschwankungen im Mikrovoltbereich, welche aus der synchronen elektrischen Aktivität von Neuronen resultieren [3]. Die Messung erfolgt über Elektroden, die auf der Kopfhaut platziert werden und die Potenzialdifferenzen zwischen verschiedenen Punkten erfassen [1].

EEG-Signale haben die Eigenschaft, extrem gerige Amplitude im Mikrovoltbereich zu besitzen, weshalb es bei der Erfassung systematisch verstärkt werden muss [1, 3]. Das Frequenzspektrum erstreckt sich dabei meist von 0,1 Hz bis 100 Hz [1].

In seiner Rohform ist ein EEG-Signal stark von Rauschen geprägt, weshalb es ein unzureichendes Signal-Rausch-Verhältnis (SNR) aufweist [3]. Da die erwarteten neuronalen Aktivitäten schwach sind, werden sie schnell von amplitudenstarken physiologischen Artefakten (wie z. B. Muskel- oder Augenbewegungen) sowie technischen Störeinflüssen überlagert, was zu Verzerrungen des Originalsignals führt [1].

2.2 Frequenzbänder

Frequenzbänder sind Frequenzbereiche, in welche die Gehirnwellen nach ihrer Frequenz eingeteilt werden. Die elektrischen Schwingungen werden in Hertz (Hz) gemessen, also in Ausschlägen pro Sekunde [1]. Die gezielte Auswertung spezifischer Frequenzbänder dient somit der fundierten Beurteilung von Gehirnaktivitäten [1].

Das EEG-Spektrum wird in jeweils 5 Frequenzbänder unterteilt [1]:

Tabelle 2.1: Die fünf EEG-Frequenzbänder und ihre assoziierten Zustände [1]

Band	**Frequenzbereich**	**Assoziierter Zustand**
Delta	0,5–4 Hz	Tiefer, sehr entspannter und erholsamer Zustand
Theta	4–8 Hz	Leichte Schlafphase oder Meditation; Zustand zwischen Schlaf und Wachsein
Alpha	8–12 Hz	Ruhiger, stressfreier Zustand; in der Großhirnrinde deutlich nachweisbar, am ausgeprägtesten bei geschlossenen Augen
Beta	12–30 Hz	Aktives Denken und Problemlösen
Gamma	ab 30 Hz	Konzentration und Multitasking

2.3 Frequenzanalyse

Wichtig ist es zu bestimmen, welche Frequenzbänder in EEG-Signalen wie stark vertreten sind. Im Folgenden wird beschrieben, welche Verfahren dafür nötig sind.

2.3.1 Fourier-Transformation

Die Fast Fourier Transform (FFT) ist ein mathematisches Verfahren, das komplexe Signale in ihre sinusförmigen Bestandteile zerlegt und sie vom Zeitbereich in den Frequenzbereich überführt [1]. Dies wird verwendet um bei der Analyse der Gehirnströme, die spektralen eigenschaften von signalen zu untersuchen und Frequenspitzen (Peaks) zu berechnen. [1].

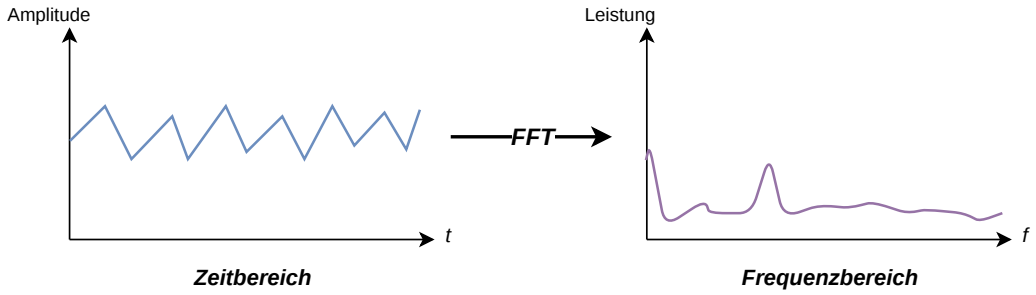


Abbildung 2.1: Überführung eines Signals vom Zeit- in den Frequenzbereich mittels FFT [19].

Das Ergebnis der FFT ist je Frequenz-Bin eine komplexe Zahl, bestehend aus einem Real- und einem Imaginärteil. Der Realteil beschreibt die Amplitude des Kosinus-Anteils und der Imaginärteil die Amplitude des Sinus-Anteils der jeweiligen Frequenz. Indem man, wie in der folgenden Gleichung, die Summe der quadrierten Real- und Imaginärteile berechnet, erhält man die Leistung eines Bins [26].

$$P = \mathrm{Re}^2 + \mathrm{Im}^2 \tag{2.1}$$

Ein zentrales Merkmal der FFT ist die Frequenzauflösung Δf, die angibt, wie genau die einzelnen Frequenzstützstellen unterteilt werden können. Die Auflösung ergibt sich aus der Abtastrate f_s und der Anzahl der Werte N pro Fenster [21]:

$$\Delta f = \frac{f_s}{N}. \tag{2.2}$$

Je größer das Fenster N gewählt wird, desto kleiner wird Δf und desto feiner ist folglich die Auflösung [20].

2.3.2 Fensterung und spektrale Leckage

Die Fensterung ist ein Verfahren, um Signale zu analysieren, deren Frequenzen sich im Laufe der Zeit verändern [1]. Die Auswertung geschieht in zeitlich verschiebbaren Abschnitten (den Fenstern) statt im gesamten Signal auf einmal [1]. Wenn ein Signal für die Datenverarbeitung abgeschnitten wird, entsteht eine spektrale Leckage. Dies geschieht häufig dann, wenn die Länge des erfassten Signalabschnitts kein exaktes Vielfaches der eigentlichen Signalperiode ist. Durch dieses harte Abschneiden wird die Energie des Signals im Frequenzspektrum verfälscht und auf einen breiteren Frequenzbereich verteilt, was zu Störfrequenzen führt. Um diese Leckage auszugleichen, nutzt man die Fensterfunktion, bei der man die Stärke des Signals an den Rändern des Abschnitts sanft abschwächt [20].

2.3.3 Welch-Methode

Um Frequenzspektren robuster zu schätzen und Rauschen zu reduzieren, wird die Welch-Methode verwendet [20]. Es ist ein Rechenverfahren, das in drei Schritten vorgeht:

1. Das ursprüngliche Signal wird in mehrere kürzere Abschnitte unterteilt, die sich teilweise überlappen dürfen [20].
2. Auf jeden Abschnitt wird eine Fensterfunktion angewendet, um dann für jeden Abschnitt einzeln das Spektrum zu berechnen [20].
3. Ganz am Ende werden die Einzelspektren zu einem Durchschnittswert gemittelt [20].

2.4 Lokale Sprachmodelle auf mobilen Geräten

Im folgenden Abschnitt werden die relevanten Grundlagen dargestellt, die für das Ausführen von Sprachmodellen auf mobilen Geräten notwendig sind. Zunächst werden große Sprachmodelle, deren Vor- und Nachteile sowie die Quantisierung vorgestellt.

2.4.1 Large Language Models

Large Language Models (LLMs) haben die Verarbeitung natürlicher Sprache (Natural Language Processing) grundlegend transformiert. LLMs sind hochkomplex und basieren auf der Transformer-Architektur [29]. Das Besondere daran ist der Self-Attention-Mechanismus (Selbstaufmerksamkeit). Dieser dient dazu, globale Abhängigkeiten zwischen Ein- und Ausgabesequenzen effizient zu modellieren [27]. Die hohe Leistungsfähigkeit der Modelle wird durch die gelernten Werte von Milliarden von Parametern erreicht. Ihnen werden herausragende Eigenschaften im Sprachverstehen, in der Textsynthese, im logischen Schlussfolgern sowie in der generativen Texterstellung verliehen [29]. Dadurch entstehen Herausforderungen, die es zu bewältigen gilt, etwa wenn man sie auf ressourcenbeschränkten Endgeräten wie Smartphones (sogenannten Edge-Geräten) zum Laufen bringen möchte. So benötigen LLMs enorm viel Speicherplatz und sehr viel Rechenleistung [29].

In der maschinellen Sprachverarbeitung werden Texte in Token zerlegt, die als kleinste Informationseinheiten dienen. So wird im Fachartikel [17] gesagt: „Ein Token kann für ein einzelnes Zeichen, einen Teil eines Wortes (z.,B. ein Suffix oder Präfix), ein ganzes Wort oder sogar eine kurze, aus mehreren Wörtern bestehende Phrase stehen.“ Damit die benötigte Rechenleistung reduziert wird, enthält jedes Token eine eindeutige numerische Identifikationsnummer (ID), anhand derer das Modell die Sprache lernt und verarbeitet [17].

Das Kontextfenster ist die maximale Anzahl an Token, die das Modell gleichzeitig verarbeiten kann [17]. Es ist sozusagen das Arbeitsgedächtnis des Modells. Sollte das Kontextfenster überschritten werden, werden die älteren Informationen gekürzt oder zusammengefasst, damit das Modell weiterhin im selben Kontext arbeiten kann [17].

2.4.2 On-Device-Inferenz und Quantisierung

On-Device-Inferenz beschreibt die lokale ausführung von KI-Modellen direkt auf den lokalen Endgeräten (beispielsweise einem Smartphone), anstatt die Daten zur Verarbeitung an einen externen Cloud-Server zu senden [29].

Dieser Ansatz bietet daher drei wesentliche Vorteile:

- **Datenschutz:** Da die Daten das Gerät nicht verlassen, wird die Privatsphäre der Nutzer besser geschützt [29].
- **Offline-Fähigkeit:** Auch ohne Internetverbindung ist die Anwendung nutzbar [29].
- **Kostenersparnis:** Es fallen keine Kosten durch die Abhängigkeit von Cloud-Diensten an [29].

Neben den Vorteilen gibt es auch gewisse Herausforderungen, die man zu bewältigen hat:

- **Speicher und Rechenleistung:** Da LLMs sehr große Sprachmodelle sind, benötigen sie aufgrund ihrer hohen Anzahl an Parametern ausreichend großen Arbeitsspeicher (RAM) und Rechenkapazität. Oft ist dies mehr, als herkömmliche Smartphones bieten, was im Extremfall zu einem Speicherüberlauf (englisch memory overflow, geläufig auch Out of Memory, OOM) führt [14, 29]. Ein OOM bezeichnet den Zustand, in dem kein Arbeitsspeicher mehr für neue Objekte verfügbar ist [14].
- **Vielfalt der Endgeräte:** Verschiedene Endgeräte besitzen unterschiedliche Prozessoren und Spezialchips. Eine Lösung zu programmieren, die auf allen Endgeräten funktioniert, ist sehr schwierig [29].
- **Katastrophales Vergessen:** Ein häufiges und bekanntes Phänomen ist, dass ein Modell unweigerlich zuvor erlerntes Wissen vergisst, wenn es neue Informationen lernt [29].

Damit diese Modelle auf Mobilgeräten laufen können, ist die sogenannte Quantisierung ein essenzielles Verfahren. Die Dateigröße des Modells wird reduziert, indem die Genauigkeit der gespeicherten Zahlenwerte künstlich verringert wird. Wechselt man beispielsweise von einer 32-Bit- auf eine 4-Bit-Speicherung, sinkt der Arbeitsspeicherbedarf und es kommt zu schnelleren Berechnungen auf dem Endgerät [29].

2.4.3 Modell und Laufzeitumgebung

Der Unterschied zwischen einem Modell und einer Runtime (Laufzeitumgebung) lässt sich grundlegend wie folgt beschreiben:

- **Modell:** Ein Modell ist die Basis, die durch das Training Fähigkeiten wie Textverständnis, logisches Schließen oder die Generierung von Inhalten erlernt hat [29].
- **Runtime:** Die Runtime ist hingegen die Software, die das Modell auf dem Endgerät ausführt [12, 29]. Sie stellt die notwendigen Ressourcen und Schnittstellen bereit, um das Modell zu betreiben [29].

2.4.4 Systemintegrierte und frei deploybare Modelle

Grundsätzlich gibt es zwei Kategorien lokaler Sprachmodelle die unterschieden werken können:

- **Systemintegrierte Modelle:** zum einen gibt es systemintegrierte Modelle, die tief in das Betriebssystem integriert sind. Ein Beispiel hierfür ist das Modell Gemini Nano, welches über den Android-Systemdienst AICore ausgeführt und lokal verwaltet wird [11]. Der architektonische Vorteil dieses Ansatzes besteht darin, dass Entwickler keine eigene Runtime bereitstellen müssen, sondern über standardisierte Programmierschnittstellen (APIs) auf das Modell zugreifen können [11]. Das Betriebssystem übernimmt dabei die zentrale Verwaltung sowie Aktualisierung, was den Speicher- und Ressourcenbedarf der einzelnen Applikation erheblich reduziert.
- **Frei deploybare Modelle:** zum anderen gibt es frei deploybare Modelle, die gemeinsam mit einer passenden Runtime als Paket von Entwicklern direkt auf das Endgerät installiert und in der Applikation ausgeführt werden müssen [16]. Dadurch kann eine maximale Flexibilität gewährleistet werden, da sie unabhängiger von spezifischen Systemvorgaben oder vorinstallierten Systemdiensten funktionieren. Für die technische Umsetzung werden sogenannte „Task Bundles“ (Dateiformate wie .task oder .litertlm) genutzt [12]. Diese Datenpakete bündeln das eigentliche Modell mit den notwendigen Tokenizer-Parametern (Werkzeuge zur Textzerlegung), um eine reibungslose Ende-zu-Ende-Verarbeitung zu gewährleisten [12]. Dieser Ansatz erhöht jedoch den Speicherbedarf der Anwendung.

2.5 Brain-Computer-Interface (BCI)

Ein Brain-Computer-Interface (BCI), zu Deutsch Gehirn-Computer-Schnittstelle, beschreibt eine Technologie, die eine Kommunikation zwischen Gehirn und Computer (externem Gerät) ermöglicht [3]. Häufig gemessen durch EEG, werden mentale Zustände und Absichten in Echtzeit erkannt. Dadurch sind Steuerungsbefehle möglich, die ganz ohne Muskelkraft auskommen [3].
Ein funktionsfähiges BCI-System erfasst die Gehirndaten, verarbeitet und analysiert diese Signale sofort in einem geschlossenen System. Der Nutzer erhält basierend auf der Klassifizierung dieser Signale eine unmittelbare Rückmeldung [3].

Vor allem körperlich stark beeinträchtigte Menschen profitieren von BCI-Systemen. Zu den bekannten Beispielen zählen [1, 3]:

- **Kommunikationssysteme:** Spezielle Systeme ermöglichen gelähmten Personen die Kommunikation mit ihrer Umwelt.
- **Bewegung und Steuerung:** BCI-Systeme erlauben die gedankliche Steuerung von Prothesen, Rollstühlen sowie das Steuern eines Cursors auf dem Computerbildschirm.
- **Assistenzsysteme:** Auch bei der Fehlerüberwachung können anhand der Gehirnaktivität die Absichten einer Notbremsung erkannt und ausgeführt werden, noch bevor das Bremspedal physisch betätigt wird.

Diese Arbeit setzt auf einem solchen EEG-basierten BCI auf und ergänzt es um ein lokales Sprachmodell, das die Messwerte in verständlichen Text überführt. Im folgenden Kapitel werden Modelle und Software-Frameworks vorgestellt, die für den mobilen Einsatz optimiert sind.

3 Stand der Technik

Dieses Kapitel gibt einen Überblick über den aktuellen Stand der Technik. Es werden zunächst die Grundlagen der On-Device-Inferenz-Lösungen erläutert, anschließend die Gemma-Modellfamilie und zum Abschluss die Ausgangslage der bestehenden Anwendung.

3.1 On-Device-Inferenz-Lösungen

Um ein Sprachmodell lokal auf einem Mobilgerät auszuführen, wird eine Laufzeitumgebung (Runtime) benötigt. Dafür stehen unterschiedliche Lösungen zur Verfügung. Diese lassen sich, wie in Abschnitt 2.4.4 dargestellt, in systemintegrierte und frei deploybare Ansätze einteilen. Beide werden im Folgenden betrachtet.

3.1.1 Systemintegrierte Lösung

Das systemintegrierte Modell Gemini Nano ist stark an die Hardware gebunden, da AICore spezifische On-Device-Hardwarekomponenten nutzt, um die Inferenz zu beschleunigen [11]. Aus diesem Grund setzt Gemini Nano bestimmte von Google freigegebene Hardware voraus. Das für diese Arbeit verwendete Testgerät Samsung Galaxy S23 erfüllt diese Anforderungen nicht, sodass Gemini Nano nicht genutzt werden kann [15].

Die Möglichkeit einer virtuellen Testumgebung wie eines Android-Emulators ist ausgeschlossen, da die physische Präsenz der Gerätehardware benötigt wird [11, 29].

Aufgrund dieser Einschränkungen steht die systemintegrierte Lösung Gemini Nano für diese diese Arbeit nicht zur Verfügung. Aus diesem Grund wird im Folgenden die frei deploybare Lösung betrachtet, die auf dem Testgerät Samsung Galaxy S23 genutzt werden kann.

3.1.2 Frei deploybare Lösung

Die MediaPipe LLM Inference API ist eine geeignete frei deploybare Lösung [12].

Abhängig vom Modelltyp und den Einstellungen können die Berechnungen entweder auf dem Hauptprozessor (CPU) oder dem Grafikprozessor (GPU) des mobilen Geräts durchgeführt werden [12].

Die Schnittstelle unterstützt von Haus aus kompakte, speziell für mobile Geräte entwickelte Modelle. Dazu gehört insbesondere die Gemma-Familie von Google, etwa Gemma 3 1B und Gemma 2 2B [12]. Darüber hinaus lassen sich über die LiteRT-Community vorkonvertierte Modelle im MediaPipe-Format einbinden [12], darunter Qwen 2.5 1.5B [22]. Für die folgenden Abschnitte sind vor allem diese drei sehr kompakten Modelle wichtig.

3.2 Die Gemma-Modellfamilie

Die Gemma-Modellfamilie gehört zu den frei deploybaren und ressourcenschonenden Modellen, die vom Gemma-Team bei Google DeepMind entwickelt wurden [8].

Besondere Merkmale der Gemma-Modelle sind:

- **Offenheit und Effizienz:** Neben der freien Zugänglichkeit der Modelle zeichnen sie sich durch eine ressourcenschonende Ausführung auf herkömmlicher Standardhardware wie Smartphones aus [7, 8].
- **Skalierbarkeit:** Um unterschiedlichen technischen Anforderungen gerecht zu werden, sind die Modelle in mehreren Größen verfügbar. Diese werden in Milliarden von Parametern gemessen. So bietet die zweite Generation (Gemma 2) Varianten mit 2, 9 und 27 Milliarden Parametern [7]. Die dritte Generation (Gemma 3) erweitert diese Skalierung auf Modelle mit 1, 4, 12 und 27 Milliarden Parametern [8]. Die aktuellste Generation (Gemma 4) bietet sogar Modelle mit bis zu 31 Milliarden Parametern [9].
- **Erweiterter Funktionsumfang:** Modelle vor der dritten Generation (Gemma 1–2) unterstützten nur reine Textverarbeitung. Seit der dritten Generation sind auch multimodale Eingaben möglich, sodass neben Text auch Bilder verarbeitet werden können [8]. Zudem wurden die Fähigkeiten zur Verarbeitung unterschiedlicher Sprachen sowie zum Erfassen deutlich längerer Textzusammenhänge verbessert [8]. Die neueste Version Gemma 4 unterstützt noch größere Kontextfenster von bis zu 256.000 Tokens und besitzt einen integrierten „Thinking-Modus“ für logisches Schlussfolgern [9].
- **Einsatzbereitschaft:** Neben der allgemeinen Basisversion gibt es die Modelle auch in einer speziell angepassten Form, die gezielt für das Führen von Dialogen und das präzise Befolgen von Anweisungen trainiert wurde [7, 8].

3.3 Die Qwen-Modellfamilie

Als drittes Modell wird Qwen des Herstellers Alibaba herangezogen. Besondere Merkmale der Qwen-Modelle sind:

- **Offenheit:** Qwen gehört ebenso zu den frei deploybaren Modellfamilien [23].
- **Skalierbarkeit:** Die Generation Qwen2.5 ist in fein abgestuften Konfigurationen von 0,5 bis 72 Milliarden Parametern verfügbar [23]. Die kompakten Varianten sind dabei gezielt für ressourcenbeschränkte Edge-Szenarien und mobile Endgeräte konzipiert [23].
- **Erweiterter Funktionsumfang:** Die neueste Generation der Familie ist Qwen3, die eine hybride Integration eines „Thinking-Modus“ sowie eines „Non-Thinking-Modus“ für schnelle Antworten innerhalb eines einheitlichen Modells bereitstellt [24].

3.4 Bestehende Anwendung

Im Folgenden wird die bestehende Anwendung vorgestellt, die als Grundlage für die Implementierung der neuen Funktionalität dient.

3.4.1 Unicorn Hybrid Black

Das Unicorn Hybrid Black ist die BCI-Plattform, die in dieser Arbeit verwendet wird. Er ist eine mobile Brain-Computer-Interface-Plattform der Firma g.tec [6]. Das Aufnahmegerät erfasst die elektrische Aktivität des Gehirns über acht Elektroden, die auf der Kopfhaut angebracht werden. Die Abtastrate beträgt genau 250 Hz [5]. Besonders ist vor allem die Kompaktheit des Geräts. Durch eine tragbare, drahtlose Bauform eignet er sich für den mobilen Einsatz. Die in Abbildung 3.1 markierten Komponenten des Geräts wurden für die Messungen dieser Arbeit verwendet.

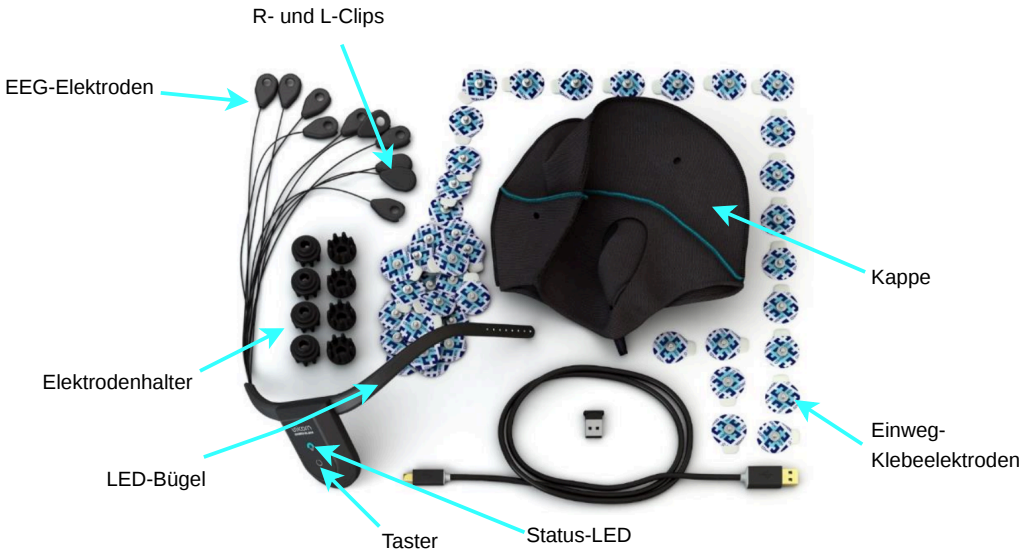


Abbildung 3.1: Komponenten des verwendeten EEG-Geräts g.tec Unicorn Hybrid Black (eigene Beschriftung, erstellt mit draw.io) [6, 19].

3.4.2 Datenübertragung und Anbindung

Die Übertragung der Messdaten an das Endgerät erfolgt drahtlos über Bluetooth, wobei jedes Sample aus den jeweils folgenden 17 Werten besteht [4].

Tabelle 3.1: Aufbau eines Samples des Unicorn Hybrid Black [4]

Werte	Bezeichnung	Anzahl
1–8	EEG-Kanäle	8
9–11	Beschleunigungssensor (x, y, z)	3
12–14	Gyroskop (x, y, z)	3
15	Batteriestand	1
16	Counter (fortlaufender Zähler)	1
17	Validation Indicator (valid-Flag)	1
	Gesamt	**17**

3.4.3 Funktionsumfang der Anwendung

Die als Grundlage für diese Arbeit genutzte Anwendung ist das Ergebnis eines Projektes aus einem vorangegangenen Praktikum. Sie bildet deren Basis, indem sie eine Verbindung mit dem Unicorn Hybrid Black aufbaut, EEG-Daten aufnimmt und speichert. Diese Daten werden zugleich als Zeitverlauf oder als Live-FFT visualisiert. Die Daten werden dauerhaft gespeichert und können zusätzlich als .db- oder .json-Datei exportiert werden. Der Datenpfad verläuft dabei vom Gerät über Bluetooth zu einem Hintergrunddienst, der die eingehenden Samples in der Datenbank ablegt.

3.4.4 Beitrag dieser Arbeit

Die bestehende Grundlage wird um neue Funktionalitäten erweitert. Die grundsätzliche Erweiterung ist die Verarbeitung der EEG-Daten durch lokale Sprachmodelle. Der Beitrag umfasst in diesem Fall drei neue Kernkomponenten:

- Schicht zur Merkmalsextraktion (`features`).
- Anbindung eines oder mehrerer Sprachmodelle (`llm`).
- Benutzeroberfläche zur KI-gestützten Analyse (`analyse`).

Darüber hinaus wird für die in der Evaluation benötigten Messungen ein Benchmark implementiert, bei dem systematisch und automatisiert gemessen wird (`benchmark`). Im Originalcode wurden nur minimale Änderungen vorgenommen.

4 Methodik

In diesem Kapitel geht es um die Methodik der Arbeit. Dabei wird als Erstes die Merkmalsextraktion näher erläutert, bei der die rohen EEG-Daten in kompakte Kennzahlen überführt werden. Anschließend werden die Modellauswahl beschrieben und die Anwendungsszenarien definiert. Abschließend folgt das Messprotokoll, das die Vorgehensweise der Messungen und die erfassten Metriken beschreibt.

4.1 Merkmalsextraktion

Die Merkmalsextraktion ist ein zentraler Bestandteil der Arbeit. Ein Sprachmodell kann keine EEG-Rohdaten verarbeiten, da bei einer Abtastrate von 250 Hz und acht Kanälen bereits in kurzer Zeit eine große, unstrukturierte Zahlenmenge entsteht. Bei einer nur einminütigen Aufnahme ergeben sich beispielsweise

$$\underbrace{8}_{\text{Kanäle}} \times \underbrace{250}_{\text{Hz}} \times \underbrace{60}_{\text{Sekunden}} = 120\,000 \text{ Werte.} \tag{4.1}$$

Diese Datenmenge muss zunächst in eine kompakte Form umgewandelt werden, weil sie ansonsten den Kontext des Modells überschreitet und keine fachliche Bedeutung transportiert. Die daraus resultierenden aggregierten Merkmale sind interpretierbar, deutlich token-sparsamer und können vom Sprachmodell verarbeitet werden.

4.1.1 Segmentierung

Bei einem Mittelwert über die gesamte Aufnahme würden Informationen wie der zeitliche Verlauf der Session fehlen. Aus diesem Grund wird jede Session in mehrere gleich große Segmente aufgeteilt (siehe Abbildung 4.1). Da das Signal mit einer konstanten Abtastrate von 250 Hz aufgenommen wird, entspricht eine gleiche Anzahl an Samples automatisch einer gleichen Zeitdauer. Die Segmentierung wird daher Sample-basiert durchgeführt. Dadurch ist es leichter nachvollziehbar, wie sich die Signalcharakteristik zwischen Beginn und Ende der Aufnahme verändert. Durch dieses Verfahren wird die Analyse robuster gegenüber Artefakten, da diese nur einen Teil der Session betreffen und nicht die gesamte Aufnahme verfälschen.

Damit die Aufteilung der Session möglich ist, muss die Aufnahme eine Mindestlänge haben. Damit pro Segment mindestens ein vollständiges Analysefenster für die spektrale Berechnung zur Verfügung steht, sind bei vier Abschnitten und einer Fenstergröße von 256 Werten mindestens

$$4 \text{ Abschnitte} \times 256 \text{ Werte (Fenster)} = 1024 \text{ Samples} \approx 4 \text{ s} \tag{4.2}$$

erforderlich. Wenn die Aufnahme zu kurz ist, um die Mindestlänge zu erfüllen, wird die Analyse abgebrochen und eine entsprechende Meldung angezeigt.

Die Fenstergröße ist kein fester, unveränderlicher Wert, sondern variabel einstellbar. Eine größere Fenstergröße erhöht die Frequenzauflösung [20], erfordert jedoch eine längere

Mindestdauer der Session. Für alle folgenden Untersuchungen wird mit der Standardfenstergröße von 256 Samples gearbeitet, da sie in etwa eine Sekunde abbildet.

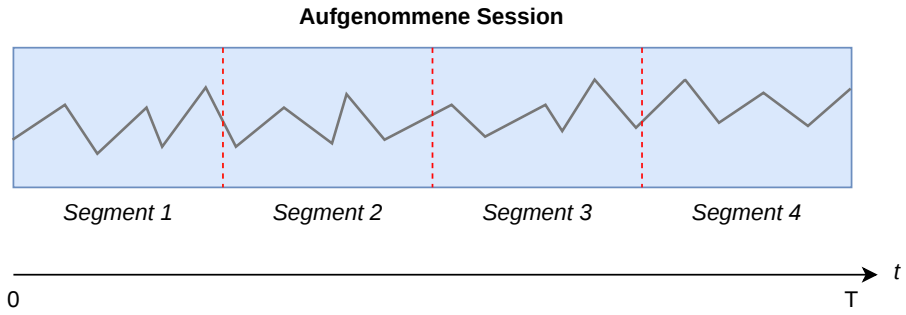


Abbildung 4.1: Aufteilung einer Session in vier gleich lange Zeitabschnitte [19].

4.1.2 Spektrale Merkmale

Die spektralen Merkmale werden für jeden Abschnitt bestimmt, indem die Signalenergie auf die einzelnen Frequenzbänder verteilt wird. Die kanalweise Berechnung erfolgt dabei mit der Welch-Methode [20]. Hierfür wird das Signal eines Abschnitts in überlappende Fenster unterteilt; im vorliegenden Fall beträgt die Fenstergröße (Standard: 256) Samples (siehe Abbildung 4.2). Für jedes dieser Fenster wird das Leistungsspektrum berechnet, das anschließend über alle Fenster gemittelt wird. Durch diese Mittelung wird das resultierende Spektrum deutlich stabiler und weniger rauschanfällig als bei einer einzelnen FFT. Allerdings ist dann mit einer geringeren Frequenzauflösung zu rechnen [20].

Um den Gleichanteil (0 Hz) des Signals zu eliminieren, wird vor der Fourier-Transformation zunächst der Mittelwert von jedem Segment subtrahiert. Anschließend wird die Hann-Fensterfunktion (auch Hanning genannt) verwendet, um jedes Fenster zu gewichten [20]. Die Gewichtung wird verwendet, da es ansonsten zu einem direkten, abrupten Abschneiden des Signals kommt, was zu harten Übergängen an den Fenstergrenzen führt [20]. Durch diesen Vorgang entsteht eine spektrale Leckage. Die Hann-Funktion hilft gegen diese Leckage, indem sie die Amplitudenwerte an den Rändern des Fensters sanft abschwächt [20].

Die Fensterfunktion wird mathematisch wie folgt beschrieben:

$$w(n) = 0.5 - 0.5 \cos\left(\frac{2\pi n}{N-1}\right) \tag{4.3}$$

wobei N die Fenstergröße (Standard: 256) bezeichnet [20].

Jedes Fenster wird per FFT in den Frequenzbereich überführt, um das jeweilige lokale Leistungsspektrum (Periodogramm) zu berechnen [1, 20]. Die Leistung der einzelnen Frequenz-Bins wird den fünf Frequenzbändern Delta, Theta, Alpha, Beta und Gamma zugeordnet [1]. Dies erfolgt durch das Aufsummieren der Leistung aller Frequenz-Bins innerhalb der jeweiligen Bandgrenzen. In der Regel schwanken die absoluten Leistungswerte stark und sind schwer interpretierbar. Daher wird die relative Bandleistung verwendet, die sich aus dem Verhältnis der Leistung eines Frequenzbandes zur Gesamtleistung ergibt. Dadurch lässt sich unmittelbar und robust ablesen, welches Frequenzband im untersuchten Abschnitt dominiert [1].

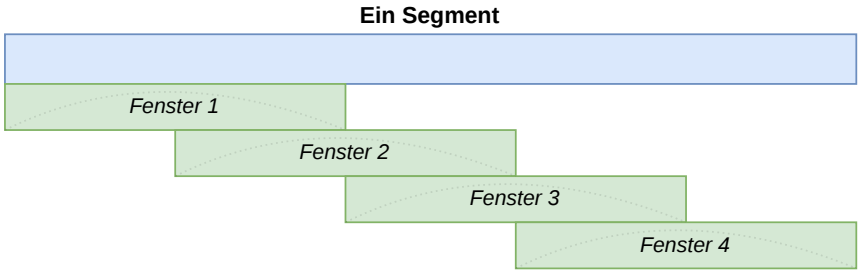


Abbildung 4.2: Zerlegung eines Segments in überlappende, mit einer Hann-Funktion gewichtete Analysefenster (50 % Überlappung) für die Welch-Methode [19].

4.1.3 Zeitstatistische Merkmale

Neben den spektralen Merkmalen gibt es auch die zeitstatistischen Merkmale. Sie werden für jeden Abschnitt berechnet [1]. Hierfür ist keine FFT notwendig, da die Merkmale direkt aus dem Zeitsignal extrahiert werden [1]. Ein zentrales Merkmal ist die Spannweite (Peak-to-Peak-Amplitude), welche die Differenz zwischen dem größten und dem kleinsten Amplitudenwert beschreibt [18]. Der Vorteil dieser Herangehensweise ist, dass sie im Gegensatz zur Betrachtung eines absoluten Maximalwertes die Unabhängigkeit von einem potenziellen Gleichanteil des Signals gewährleistet. Aus diesem Grund stellt die Peak-to-Peak-Amplitude ein robustes Kriterium dar, um extreme Signalausschläge und Drifts zu identifizieren, wie sie bei Artefakten auftreten können [18].

Des Weiteren wird zur Beurteilung der Signalqualität der Anteil der ungültigen Messwerte betrachtet. Dies ist der Anteil der Datenpunkte, die vom Erfassungsgerät selbst als fehlerhaft markiert wurden. Man kann durch diesen wichtigen Indikator erkennen, ob Messfehler vorliegen, wie beispielsweise ein unzureichender Kontakt an der Kopfhaut [1].

Im Allgemeinen bilden statistische Merkmale ergänzend zu den spektralen Merkmalen eine gute Grundlage, um die Datenqualität zu beurteilen und Artefakte zu erkennen.

4.1.4 Aufbereitung für das Sprachmodell

Im letzten Schritt der Merkmalsextraktion werden die berechneten Merkmale in eine textuelle Form gebracht, die vom Sprachmodell verarbeitet werden kann. Dabei enthält der Text folgende Informationen:

- aggregierte Kennzahlen
- relative Bandleistung der fünf Frequenzbänder (pro Abschnitt)
- Spannweite (Peak-to-Peak-Amplitude)
- Anteil der ungültigen Messwerte

Dabei werden bewusst keine Bewertungen oder Interpretationen in den Text aufgenommen. Alle Informationen sind objektive Messwerte, deren Bedeutung allein dem Sprachmodell überlassen wird. Dadurch wird gewährleistet, dass die eigentliche Analysearbeit vom Modell erledigt wird.

4.2 Modellauswahl

Wie schon in Abschnitt 3.1.1 beschrieben, stehen systemintegrierte Modelle auf dem verwendeten Testgerät nicht zur Verfügung. Daher konzentriert sich diese Arbeit auf frei deploybare Modelle, wie sie in Abschnitt 3.2 und Abschnitt 3.3 beschrieben sind.

Um eine geeignete Evaluierung zu ermöglichen, wird der Einfluss der Modellgröße unter konstanten Ausführungsbedingungen untersucht. Dazu werden drei Modelle vergleichend betrachtet: zum einen die kompakte Variante Gemma 3 1B [8, 12], zum anderen die parameterstärkere Gemma 2 2B als größeres Referenzmodell [7, 12], sowie das Modell Qwen 2.5 1.5B des Herstellers Alibaba [12, 23], dessen Größe zwischen den beiden Gemma-Modellen liegt. Da alle drei Modelle nativ über die MediaPipe-Runtime (LLM Inference API) laufen [12], lässt sich bei gleicher Laufzeitumgebung sowohl der Effekt der Modellgröße als auch der Einfluss des Herstellers auf die Systemperformanz untersuchen.

Alle drei Modelle werden in 8-Bit-Quantisierung und mit identischer Kontextgröße (1280 Token) ausgeführt, sodass sich der Vergleich einzig auf die Modellgröße und den Hersteller bezieht. Zudem wird die Token-Auswahl über einen festen Top-K-Wert von 64 begrenzt, was bedeutet, dass immer nur die 64 wahrscheinlichsten Token als Kandidaten betrachtet werden.

4.3 Anwendungsszenarien

Um die Eignung der Modelle für die Analyse von EEG-Daten zu untersuchen, werden drei Anwendungsszenarien betrachtet. Dabei wird untersucht, wie ein Sprachmodell die extrahierten Merkmale einer oder mehrerer Sessions verarbeitet. Jedes Szenario besteht aus einem einmaligen Aufruf des Modells, der die Merkmale der Session(s) als Eingabe erhält. Es wird keine vorab bekannte Musterlösung benötigt.

- **Session-Report:** Aus den Merkmalen einer Session wird ein zusammenhängender, lesbarer Bericht generiert.
- **Artefakt-Erklärung:** Das Modell erhält die Kennzahlen einer Session und erklärt auffällige Muster (z.B. hohe Peak-to-Peak-Amplituden oder einen hohen Anteil ungültiger Werte) in natürlicher Sprache.
- **Session-Vergleich:** Es werden die Merkmale zweier Sessions übergeben und darauffolgend verglichen.

Der Prompt-Aufbau ist in allen drei Szenarien und Modellen identisch. Er beginnt mit der szenariospezifischen Anweisung, gefolgt von einem textuell aufbereiteten Merkmalssatz. Das Niveau steigt dabei mit jedem Szenario. Während der Report lediglich ein Zusammenfassen erfordert, ist beim Artefakt-Erklärungsszenario zusätzlich das Deuten notwendig. Das Vergleichsszenario erfordert schließlich ein Schlussfolgern über zwei Datensätze hinweg. Dadurch kann man die Grenzen der Modelle in Bezug auf die Komplexität der Aufgabenstellung untersuchen.

4.4 Messprotokoll

Eine verlässliche Messung kann nur systematisch und unter kontrollierten Bedingungen durchgeführt werden. Da es sich um mehrere Messungen handelt, werden sie in kurzer

Zeit automatisiert über alle Modell-Szenario-Kombinationen durchgeführt und in einer CSV-Datei protokolliert. So wird sichergestellt, dass die Messungen eine reproduzierbare und auswertbare Datenbasis für die anschließende Evaluation liefern. Bevor die eigentliche Messung beginnt, wird eine Aufwärmphase durchgeführt, um Kaltstart-Effekte zu vermeiden. Dadurch wird verhindert, dass die erste Konfiguration verzerrt gemessen wird.

4.4.1 Metriken

Zur Bewertung der Modelle werden mehrere Metriken erfasst, die sich in Latenz, Ressourcenverbrauch und Stabilität gliedern.

Tabelle 4.1: Übersicht der erfassten Metriken.

Kategorie	**Metrik**	**Beschreibung**
Latenz	Ladezeit	Zeit zum Laden des Modells
	TTFT	Zeit bis zum ersten erzeugten Token
	Gesamtdauer	Dauer der gesamten Inferenz
	Tokens/s	Durchsatz, unabhängig von der Ausgabelänge
Ressourcen	RAM (PSS)	Speicherverbrauch inkl. nativem Speicher
	Akku	Energieverbrauch (µAh) über eine Messreihe
Stabilität	Status	erfolgreich, Fehler oder Speicherüberlauf
Qualität	Ausgabequalität	qualitative Bewertung (siehe Kapitel 6)

Die Tokens pro Sekunde sind vor allem wichtig, da sie einen fairen Vergleich unabhängig von der Länge der Ausgabe erlauben. Eine reine Zeitmessung würde längere Antworten offensichtlich benachteiligen.

Der Speicherverbrauch wird über die Proportional Set Size (PSS) erfasst, da diese exklusiv von der Applikation genutzte Speicherseiten sowie den proportionalen Anteil an gemeinsam verwendeten Seiten (Shared Pages) berücksichtigt. Dadurch eignet sie sich zur präzisen Bestimmung des tatsächlichen Speicherbedarfs, da durch sie gemeinsam genutzte Ressourcen (wie beispielsweise Bibliotheken oder Systemdienste) nicht mehrfach gezählt werden [13].

Der Akkuverbrauch wird über den BatteryManager des Geräts vor und nach einer Messreihe ermittelt. Die Messung erfolgt in Mikroamperestunden (µAh), da diese eine feinere Auflösung als die Angabe in Prozent bietet [10].

Im Gegensatz zum Benchmark, bei dem es gleich fünf Messungen gibt, werden für die reine Analyse einer Session nur Latenzmetriken erfasst. Der Grund liegt hierbei an der Menge der Messungen. Der Energieverbrauch lässt sich nur über eine ganze Messreihe sinnvoll bestimmen, da er sich erst aus der Differenz des Akkustands über mehrere aufeinanderfolgende Durchläufe ergibt. Zudem schwankt der Arbeitsspeicherbedarf stark [13], sodass eine Messung keine verlässliche Aussage über den Speicherbedarf zulässt. Die Metriken Status und Ausgabequalität sind nur für die Evaluation relevant und werden daher nicht in der Analyse erfasst.

4.4.2 Messbedingungen

Damit die Messungen unter vergleichbaren und fairen Bedingungen erfolgen können, werden sie unter den folgenden Rahmenbedingungen durchgeführt:

- Der Flugmodus im Testgerät ist aktiviert, um Netzwerkaktivitäten auszuschließen.
- Die Bildschirmhelligkeit wird auf einen festen Wert (Ungefähr 50 %) eingestellt, um den Akkuverbrauch über alle Messungen hinweg vergleichbar zu halten.
- Das Handy befindet sich nicht im Ladezustand, da dies den Akkuverbrauch während der Messung verfälschen würde. Bei Start der Messung befindet sich der Akkustand bei ungefähr 60 %.
- Alle unnötigen Hintergrundprozesse werden beendet, um die Systemlast zu minimieren.
- Alle Messungen finden möglichst innerhalb von 20 Minuten statt, um die Temperatur des Geräts konstant zu halten.
- Alle Messungen werden auf demselben Testgerät durchgeführt, um die Hardwarebedingungen konstant zu halten. Dabei handelt es sich um ein Samsung Galaxy S23 mit dem Prozessor Qualcomm Snapdragon 8 Gen 2 und 8 GB Arbeitsspeicher, betrieben unter Android 16 (One UI 8.5), mit einer Akkukapazität von 3900 mAh [2, 25].
- Für alle Messungen werden dieselben Sessions als Eingabe verwendet, sodass alle Metriken unter identischen Bedingungen erfasst werden.
- Unmittelbar vor der Messung wurde das Testgerät für mindestens 20 Minuten nicht genutzt, damit es abkühlen konnte.
- Während der gesamten Messung bleibt das Testgerät eingeschaltet und die Session-Detailansicht geöffnet, da der Benchmark sonst unterbrochen wird.

Es wird jede Kombination von Modell und Anwendungsszenario N = 5 Mal gemessen, da Temperatur, Hintergrundlast oder Caching die Laufzeit von Lauf zu Lauf erheblich beeinträchtigen können [16]. Dabei wird die erste Messung standardmäßig verworfen, da sie durch das Initialisieren des Modells deutlich länger dauert. Die verbleibenden Werte werden gemittelt. Zusätzlich wird die Standardabweichung berechnet, die die Streuung der Messwerte um den Mittelwert beschreibt. Der Mittelwert $\bar{x}$ und die Standardabweichung s über die n einbezogenen Messwerte ergeben sich dabei wie folgt:

$$\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i \qquad s = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2} \tag{4.4}$$

Dadurch wird die Stabilität der Messungen sichtbar. Ein hoher Wert deutet auf eine starke Streuung hin, während ein niedriger Wert eine geringe Streuung und damit eine hohe Stabilität der Messungen anzeigt.

Das Verwerfen der ersten Messung, die Mittelung der verbleibenden Messungen sowie die Berechnung der Standardabweichung erfolgen nicht in der Anwendung selbst. Sie werden stattdessen manuell auf Grundlage der in der CSV-Datei gespeicherten Rohwerte durchgeführt.

5 Implementierung

Der Code wurde als native Android-App in der Programmiersprache Java umgesetzt. Android Studio wurde dabei als Entwicklungsumgebung verwendet, und der Build erfolgt über das Build-System Gradle. Die im Folgenden verwendeten Bibliotheken werden in den jeweiligen Abschnitten genannt.

5.1 Architektur

Die Implementierung der Anwendung besteht aus den folgenden Komponenten, die in Abbildung 5.1 dargestellt sind:

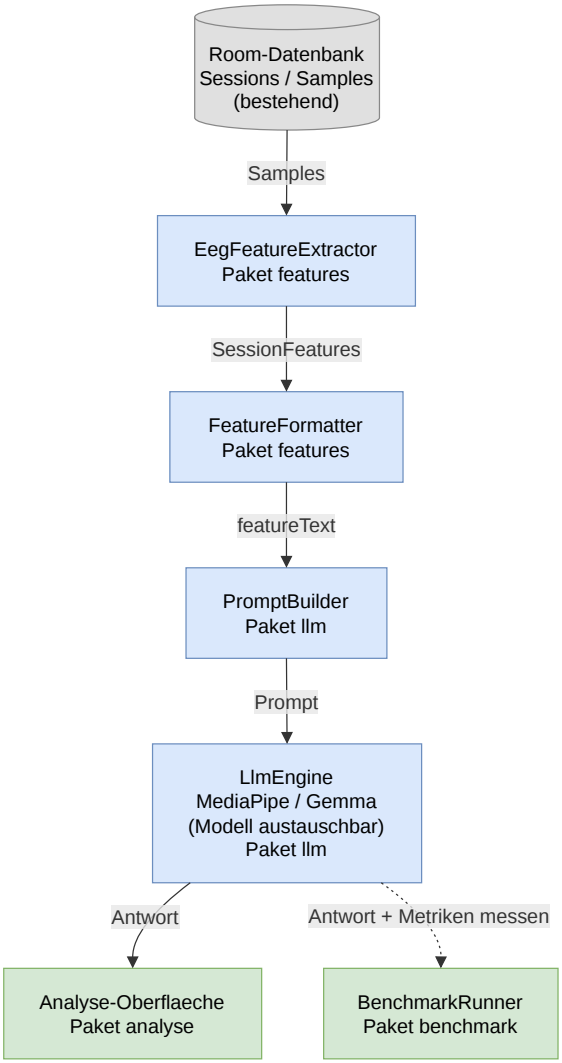


Abbildung 5.1: Architektur und Datenfluss der Anwendung [19]

- **Datenbank:** Der Weg startet über die Datenbank, aus der die Samples abgerufen werden.
- **EegFeatureExtractor:** Hier werden die Samples verarbeitet, indem Merkmale berechnet werden.
- **FeatureFormatter:** Wandelt die Merkmale in ein für das Sprachmodell verständliches Format um.
- **PromptBuilder:** Baut die Anweisung für den Prompt (je Szenario).
- **LlmEngine:** Führt das Modell (Gemma via MediaPipe) aus und gibt die Antwort zurück.
- **Analyse-Oberfläche:** Zeigt letztlich die Antwort an.

- **Benchmark:** Misst die Verarbeitung (Laufzeit, Speicherverbrauch etc.).

Die Architektur ist in drei Kernbereiche unterteilt: zum einen das Paket `features`, das die Merkmalsextraktion übernimmt; zum anderen das Paket `llm`, das je nach Szenario den Aufbau des Prompts übernimmt und das Sprachmodell ausführt; und schließlich das Paket `analyse`, das die Benutzeroberfläche bereitstellt. Durch die Trennung dieser Pakete bleibt der Code übersichtlich, wartbar und erweiterbar.

Ein zentrales Merkmal der Architektur ist die Kapselung des Sprachmodells in der Klasse `LlmEngine`. Die Klasse stellt eine einfache Schnittstelle bereit, die für das Laden, Anfragen und Freigeben verantwortlich ist. Dadurch wird die konkrete Modell- und Laufzeitumgebung vor dem Rest der Anwendung verborgen. Dies ermöglicht das Austauschen der Modelle, ohne dass die anderen Komponenten angepasst werden müssen. In dieser Arbeit werden die Modelle Gemma 3 1B und Gemma 2 2B allein durch den Wechsel des Modellpfads auf dem mobilen Gerät geladen. Aus diesem Grund ließe sich an derselben Schnittstelle auch ein anderes Modell anbinden. Möglich wäre es auch, eine andere Laufzeitumgebung oder ein systemintegriertes Modell anzubinden. Diese Entkopplung ermöglicht es, die Anwendung flexibel und ohne großen Aufwand zu erweitern und zu warten.
Die in Abbildung 5.1 grün und blau markierten Komponenten stellen den eigenen Beitrag dar.

5.2 Implementierung der Merkmalsextraktion

Die Merkmalsextraktion ist im Paket `features` implementiert; zuständig dafür ist die Klasse `EegFeatureExtractor`. Sie durchläuft für jede Session eine feste Abfolge von Schritten: Als Erstes wird die Session aus der Datenbank geladen, daraufhin wird sie auf ihre Verwertbarkeit geprüft. Anschließend wird sie in gleich große Segmente aufgeteilt, bei denen dann jeweils die spektralen sowie statistischen Merkmale berechnet werden. Die Ergebnisse werden alle im Objekt `SessionFeatures` gesammelt und schließlich vom `FeatureFormatter` in einen lesbaren Text umgewandelt, den das Sprachmodell verarbeiten kann. Im Folgenden wird chronologisch auf die einzelnen Schritte genauer eingegangen.

Vorbereitung. Sobald die Klasse erzeugt wird, werden vorab alle wiederverwendbaren Bestandteile angelegt (Listing 5.1). Dazu gehört das Hann-Fenster, das für die Fourier-Transformation vorab berechnet wird, um bei jedem Fenster nicht erneut erzeugt zu werden. Außerdem wird das FFT-Objekt der Bibliothek JTransforms für die gewählte Fenstergröße instanziiert [28]. Der Standardwert für die Fenstergröße beträgt 256, kann aber variabel angepasst werden. Aus ihr und der Anzahl der Segmente wird die Mindestlänge `minSamples` der Session berechnet.

```
public EegFeatureExtractor(int windowSize) {
    this.windowSize = windowSize;
    this.hann       = hannWindow(windowSize);
    // Hann-Fenster vorberechnen
    this.fft        = new DoubleFFT_1D(windowSize);
    // FFT-Objekt aus JTransforms
    this.minSamples = windowSize * SEGMENT_COUNT;
}
```

Listing 5.1: Vorberechnung von Hann-Fenster und FFT-Objekt im Konstruktor

Validierung. Die Methode `validate` prüft eine Session auf ihre Verwertbarkeit. Es gibt zwei Kriterien, die erfüllt sein müssen. Zum einen muss die Session mindestens so viele Samples enthalten, wie in `minSamples` berechnet wurde, ansonsten wird `TOO_SHORT` zurückgegeben. Zum anderen muss der Anteil ungültiger Samples unter 50 % liegen. Ist dies nicht der Fall, wird `TOO_MANY_INVALID` zurückgegeben. Sind beide Kriterien erfüllt, wird die Analyse mit der Rückgabe von `OK` fortgesetzt, ansonsten wird dem Nutzer eine entsprechende Meldung angezeigt.

```
public enum Check { OK, TOO_SHORT, TOO_MANY_INVALID; }

public Check validate(List<SampleEntity> samples) {
    if (samples == null || samples.size() < minSamples) {
        return Check.TOO_SHORT;
        // zu wenige Samples
    }
    int invalidCount = 0;
    for (SampleEntity sample : samples) {
        if (sample.valid == 0) invalidCount++;
    }
    if (invalidCount > MAX_INVALID_RATIO * samples.size()) {
        return Check.TOO_MANY_INVALID;
        // zu viele ungueltige Werte
    }
    return Check.OK;
}
```

Listing 5.2: Vorabprüfung der Session

Segmentierung. Die Methode `extract` stellt den Einstiegspunkt der Merkmalsextraktion dar. Dort wird die Session in `SEGMENT_COUNT` gleich große Abschnitte (hier vier) zerlegt. Für jedes dieser Segmente werden separat die Merkmale (Listing 5.3) berechnet. Die Aufteilung in gleich große Stücke hilft dabei, den zeitlichen Verlauf innerhalb einer Session grob abzubilden, anstatt sie nur als ein einziges Mittel zu betrachten. Die einzelnen Segmente werden jeweils im `SessionFeatures`-Objekt abgelegt.

```
public SessionFeatures extract(List<SampleEntity> samples) {
    SessionFeatures features = new SessionFeatures();
    features.sessionId       = samples.get(0).sessionId;
    features.sampleCount     = samples.size();
    features.durationSeconds = samples.get(samples.size() - 1)
        .sampleCounterSeconds;
    features.segments        = new ArrayList<>();

    int segLength = samples.size() / SEGMENT_COUNT;
    for (int i = 0; i < SEGMENT_COUNT; i++) {
        int start = i * segLength;
        int end   = (i == SEGMENT_COUNT - 1) ? samples.size() : start
            + segLength;
        List<SampleEntity> slice = samples.subList(start, end);
        features.segments.add(buildSegment(slice));
        // Merkmale pro Abschnitt berechnen
    }
    return features;
}
```

Listing 5.3: Zerlegung der Session in Segmente

Spektrale Merkmale (Welch-Methode). Für jeden der acht EEG-Kanäle wird die Leistung in den Frequenzbändern Delta (δ), Theta (θ), Alpha (α), Beta (β) und Gamma (γ) berechnet. Das geschieht nach dem Verfahren der Welch-Methode (Listing 5.4), bei der die Fourier-Transformation auf 50 % überlappende Segmente des Signals angewendet wird. Für jedes Fenster wird das Leistungsspektrum berechnet und in Bänder einsortiert, und über alle Fenster hinweg wird gemittelt. Durch die Mittelung wird ein stabileres Ergebnis geliefert, das weniger anfällig für zufällige Schwankungen ist.

```
private double[][] computeBandPower(List<SampleEntity> slice) {
    double[][] result = new double[CHANNELS][NUM_BANDS];
    int hop = windowSize / 2;
    // 50% Ueberlappung

    for (int i = 0; i < CHANNELS; i++) {
        double[] signal = extractChannelSignal(slice, i);
        double[] bandAccum = new double[NUM_BANDS];
        int windowCount = 0;

        for (int start = 0; start + windowSize <= signal.length; start += hop) {
            double[] window = Arrays.copyOfRange(signal, start, start + windowSize);
            double[] power  = powerSpectrum(window);
            // Fenster -> Spektrum
            double[] bands  = sumBands(power);
            // Spektrum -> Baender
            for (int b = 0; b < NUM_BANDS; b++) bandAccum[b] += bands[b];
            windowCount++;
        }
        if (windowCount > 0) {
            for (int b = 0; b < NUM_BANDS; b++) {
                result[i][b] = bandAccum[b] / windowCount;
                // ueber alle Fenster mitteln (Welch)
            }
        }
    }
    return result;
}
```

Listing 5.4: Band-Power je Kanal nach dem Welch-Verfahren

Die Methode `powerSpectrum` berechnet das Leistungsspektrum eines Fensters (Listing 5.5). Zunächst wird der Mittelwert (Gleichanteil) abgezogen. Danach wird das Hann-Fenster angewendet, um anschließend die Fourier-Transformation durchzuführen. Schließlich wird die Leistung je Frequenz-Bin berechnet, indem die Real- und Imaginärteile quadriert und summiert werden.

```
private double[] powerSpectrum(double[] window) {
    // 1. Mittelwert (Gleichanteil)
    double mean = 0;
    for (double v : window) mean += v;
    mean /= window.length;
```

```
    // 2. Mittelwert abziehen + Hann anwenden
    double[] data = new double[windowSize];
    for (int n = 0; n < windowSize; n++) {
        data[n] = (window[n] - mean) * hann[n];
    }

    // 3. FFT (in-place, JTransforms)
    fft.realForward(data);

    // 4. Leistung pro Frequenz-Bin
    double[] power = new double[windowSize / 2];
    power[0] = data[0] * data[0];
    // DC-Bin
    for (int k = 1; k < windowSize / 2; k++) {
        double re = data[2 * k], im = data[2 * k + 1];
        power[k] = re * re + im * im;
    }
    return power;
}
```

Listing 5.5: Berechnung des Leistungsspektrums eines Fensters

Die eigentliche Fourier-Transformation (`fft.realForward`) wird dabei durch die Bibliothek JTransforms bereitgestellt und nicht selbst implementiert [28]. Sie bietet eine Java-basierte Implementierung der Fourier-Transformation, die den Aufruf `realForward` für reelle Eingangssignale bereitstellt. Das Ergebnis wird dabei in dasselbe Array geschrieben, sodass die geraden Indizes die Real- und die ungeraden Indizes die Imaginärteile der Frequenz-Bins enthalten.

Zuordnung zu den Frequenzbändern. Das Spektrum liefert die Leistung je Frequenz-Bin. Diese müssen noch den Frequenzbändern zugeordnet werden (Listing 5.6). Dazu wird jeder Bin-Index über die Beziehung $\text{freq} = i \cdot f_s / N$ in eine Frequenz umgerechnet. Der Faktor f_s / N entspricht dabei der Frequenzauflösung Δf (Gleichung 2.2), also dem Frequenzabstand zwischen zwei benachbarten Bins. Dadurch können für jedes Frequenzband die entsprechenden Bins summiert werden.

```
private double[] sumBands(double[] power) {
    double[] bandSum = new double[NUM_BANDS];
    for (int i = 1; i < power.length; i++) {
        double freq = (double) (i * SAMPLE_RATE) / windowSize;
        // Bin-Index -> Frequenz
        for (FrequencyBand band : FrequencyBand.values()) {
            if (freq >= band.lowHz && freq < band.highHz) {
                bandSum[band.ordinal()] += power[i];
            }
        }
    }
    return bandSum;
}
```

Listing 5.6: Zuordnung der Frequenz-Bins zu den Bändern

Die Einteilung in die jeweiligen Frequenzbänder wird durch den Aufzählungstyp (`enum`) `FrequencyBand` realisiert, der als zentrale Definitionsstelle der Bänder dient (Listing 5.7).

Jedes Band ist dort mit seiner unteren und oberen Grenzfrequenz hinterlegt.

```
public enum FrequencyBand {
    DELTA(0.5f, 4f),
    THETA(4f,   8f),
    ALPHA(8f,   12f),
    BETA(12f,   30f),
    GAMMA(30f,  45f);

    public final float lowHz;
    public final float highHz;

    FrequencyBand(float lowHz, float highHz) {
        this.lowHz  = lowHz;
        this.highHz = highHz;
    }
}
```

Listing 5.7: Zentrale Definition der Frequenzbänder mit ihren Grenzen

Statistische Merkmale. Als erstes der zwei statistischen Merkmale wird der Peak-to-Peak-Wert berechnet. Dieser stellt die Spannweite je Kanal dar. Er beschreibt die Differenz zwischen dem höchsten und dem niedrigsten Messwert. Dadurch sind starke Ausschläge oder Drift sichtbar. Der zweite statistische Wert ist der Anteil der ungültigen Samples (`valid == 0`) an der Gesamtzahl im Segment. Er dient als Maß für die Signalqualität.

Ergebnis. Abschließend werden die berechneten Merkmale in einem Objekt `SessionFeatures` gesammelt. Dieses dient als Datencontainer, der alle Daten der Session sowie die Liste der Segmente mit ihren Merkmalen enthält. Das Ergebnis dieser Schritte ist die Grundlage für die anschließende Aufbereitung durch den `FeatureFormatter`.

5.3 Integration des Sprachmodells

Die gesamte Integration rund um das Sprachmodell ist im Paket `llm` implementiert. Dort befindet sich die Klasse `LlmEngine`, die für die gesamte Kommunikation mit dem Modell zuständig ist. Dabei hat die Klasse drei Kernaufgaben. Dazu zählen das Laden, Anfragen und Freigeben des Modells. Die MediaPipe LLM Inference API wird für die lokale Ausführung des Modells verwendet [12], die als Abhängigkeit `com.google.mediapipe:tasks-genai` eingebunden ist. Die quantisierten Modelle Gemma 3 1B, Gemma 2 2B und Qwen 2.5 1.5B liegen als `.task`-Dateien im Verzeichnis `/data/local/tmp/llm/` auf dem lokalen Gerät, wohin sie vorab per `adb` übertragen wurden. Im Weiteren wird der Lebenszyklus eines Modells in seiner Ausführungsreihenfolge beschrieben.

Modellauswahl und Pfade. Die jeweils verwendeten Modelle sind als konstante Pfade hinterlegt (Listing 5.8). Diese werden mithilfe der Methode `pathFor` selektiert. Die Auswahl des Modells geschieht zuvor, wobei die Kennung an die `LlmEngine` übergeben wird.

```
public static final String MODELL_1B   = "/data/local/tmp/llm/Gemma3-1B-IT-...q8_ekv1280.ta
public static final String MODELL_2B   = "/data/local/tmp/llm/gemma2-2b.task";
public static final String MODELL_QWEN = "/data/local/tmp/llm/Qwen2.5-1.5B-...q8_ekv1280.ta

public static String pathFor(String modell) {
    if ("2b".equals(modell))   return MODELL_2B;
```

```
    if ("qwen".equals(modell)) return MODELL_QWEN;
    return MODELL_1B;
}
```

Listing 5.8: Modellpfade und Auswahl des Modells

Laden und Konfiguration. Das Laden des Modells besteht aus dem Konfigurieren der Optionen (Listing 5.9). Hierzu werden in einem `LlmInferenceOptions`-Objekt die Pfadangabe, `maxTopK` und `maxTokens` festgelegt. Die Obergrenze der wählbaren Token wird auf 64 gesetzt. Als maximale Anzahl der Token wird 1280 festgelegt. Hierbei gelten diese Werte über alle Modelle hinweg, um einen fairen Vergleich zu bieten (Kapitel 4). Erzeugt und einsatzbereit ist das Modell anschließend nach dem Aufruf von `createFromOptions`.

```
public void load(Context context) {
    LlmInferenceOptions options = LlmInferenceOptions.builder()
            .setModelPath(modelPath)
            .setMaxTopK(64)
            // Sampling-Obergrenze
            .setMaxTokens(1280)
            // Kontextgroesse (Prompt + Ausgabe)
            .build();

    llm = LlmInference.createFromOptions(context, options);
}
```

Listing 5.9: Laden und Konfigurieren des Modells

Streamende Ausgabe. Die Antwort eines Modells erfolgt in einem Stream, bei dem die Token nach und nach geliefert werden (Listing 5.10). Der vom `PromptBuilder` erzeugte Prompt wird über `generateResponseAsync` an das Modell übergeben. Dabei wird jedes Teilstück über einen Callback gemeldet. So wird der Verlauf der Entstehung der Antwort in Echtzeit an die Benutzeroberfläche weitergeleitet. Gleichzeitig wird der Time-To-First-Token gemessen. Dieser wird mit der vollständigen Antwort samt Gesamtdauer zurückgegeben.

```
public void questionStreaming(String prompt, StreamCallback callback) {
    StringBuilder full = new StringBuilder();
    long start = System.nanoTime();
    long[] ttft = {-1};

    llm.generateResponseAsync(prompt, (partial, done) -> {
        if (ttft[0] < 0) {
            ttft[0] = (System.nanoTime() - start) / 1_000_000;
            // erstes Token -> TTFT festhalten
        }
        full.append(partial);
        callback.onChunk(partial);
        // Teilstueck weiterreichen
        if (done) {
            long totalMs = (System.nanoTime() - start) / 1_000_000;
            callback.onDone(full.toString(), ttft[0], totalMs);
        }
    });
}
```

Listing 5.10: Streamende Ausgabe mit Messung der Latenz

Freigabe. Letztendlich wird das Modell über `close` wieder freigegeben.

5.4 Prompt-Design und iterative Optimierung

Wie schon in Abschnitt 4.3 erwähnt, besteht der Prompt aus den aufbereiteten Merkmalen mit einer szenariospezifischen Anweisung. Dieser ist für alle Modelle identisch. Der Prompt wurde in der Klasse `PromptBuilder` realisiert. Der Weg bis zum finalen Prompt war iterativ: Er wurde anhand der Modellausgaben verfeinert und anschließend für die Evaluation final festgehalten.

Bei der Optimierung des Prompts ergaben sich folgende Design-Vorgaben, die jeweils auf ein beobachtetes Problem der frühen Ausgaben zurückgehen:

- **Datentreue:** Es wird darauf hingewiesen, nur die Werte zu verwenden, die genannt werden. Dies soll verhindern, dass das Modell eigene Annahmen trifft. Dieses Problem trat in früheren Ausgaben auf, so wurden Begriffe wie „Dominanzindex“ genannt.
- **Reines Textformat:** Es wird sichergestellt, dass die Ausgaben des Modells nur textuelle Informationen enthalten. Es werden Sonderzeichen (Markdown) verboten, da sonst Formatierungszeichen (z.,B. `**`) auftraten.
- **Eindeutige Merkmalsbenennung:** Im `FeatureFormatter` wird der Anteil ungültiger Werte als „Ungültige Messwerte: X %“ statt als „Qualität: X %“ ausgegeben. Zuvor interpretierte das Modell „Qualität: X %“ fälschlich als schlechte Qualität.
- **Begrenzung der Interpretation:** Ausgaben mit falscher Interpretation, etwa durch die Einbeziehung medizinischer Diagnosen, werden verhindert, da sie zuvor ein Problem darstellten.
- **Kompaktheit:** Wenige Sätze, keine Begrüßung oder Rückfragen.

Nach der Optimierung wurde der finale Prompt eingefroren und für alle Messungen verwendet. Er stellt auch die Grundlage für den Benchmark dar. In Listing 5.11 sind die finalen Prompts je Szenario dargestellt.

```
// Szenario 1: Session-Report
public String buildReport(String featureText) {
    return "Fasse die folgenden EEG-Messwerte in 3 bis 4 Saetzen
        sachlich zusammen. "
          + "Schreibe nur den Bericht: keine Begruessung, keine
              Rueckfragen, kein Schlusssatz. "
          + "Schreibe reinen Fliesstext ohne Sonderzeichen (kein
              Markdown). "
          + "Nenne das dominante Frequenzband, die Signalqualitaet und
              auffaellige Ausschlaege. "
          + "Erfinde nichts und nutze nur die folgenden Werte:\n\n"
          + featureText;
}
```

```
// Szenario 2: Artefakt-Erklaerung
public String buildArtefaktErklaerung(String featureText) {
    return "Analysiere die folgenden EEG-Messwerte und beziehe dich
        konkret auf die genannten Werte. "
          + "Benenne die auffaelligsten Punkte dieser Session (z. B.
              sehr hohe Ausschlaege, "
          + "ungueltige Daten oder ein stark dominantes Band) und
              erklaere kurz ihre moegliche "
          + "technische Ursache (z. B. Bewegung, Elektroden, Verbindung
              ). Stelle keine "
          + "medizinischen Diagnosen. Schreibe wenige Saetze als
              sachlichen Fliesstext, ohne "
          + "Begruessung und ohne Sonderzeichen (kein Markdown).
              Erfinde nichts und nutze nur "
          + "die folgenden Werte:\n\n"
          + featureText;
}

// Szenario 3: Session-Vergleich
public String buildVergleich(String featureTextA, String featureTextB)
    {
    return "Vergleiche die folgenden zwei EEG-Sessions und beschreibe
        die wichtigsten "
          + "Unterschiede in einfachen Worten (z. B. Banddominanz,
              Signalqualitaet, Ausschlaege). "
          + "Schreibe sachlichen Fliesstext ohne Sonderzeichen (kein
              Markdown). Erfinde keine "
          + "Kennzahlen oder Begriffe. Schreibe nur den Vergleich,
              keine Begruessung. "
          + "Nutze nur die folgenden Werte.\n\n"
          + "Session A:\n" + featureTextA
          + "\n\nSession B:\n" + featureTextB;
}
```

Listing 5.11: Aufbau der Prompts je Szenario (finale, eingefrorene Fassung)

5.5 Benutzeroberfläche

5.6 Benchmark

Die gesamte Logik des Benchmarks befindet sich im Paket `benchmark`. Von dort aus werden alle Messungen, die in Abschnitt 4.4 definiert sind, automatisiert durchgeführt. Dabei werden alle Kombinationen durchlaufen und in die CSV-Datei geschrieben. Die Implementierung wird in drei Klassen aufgeteilt: `BenchmarkRunner` steuert den Ablauf, `Metrics` liefert die Messwerte für Speicher und Akku, und `BenchmarkLogger` schreibt die Ergebnisse. Den Start bildet der Benchmark-Button in der Session-Detailansicht.

Ablauf. Der Hintergrund-Thread `runAll` ist für den gesamten Messlauf verantwortlich (Listing 5.12). Dieser sorgt dafür, dass die Oberfläche nicht blockiert wird. Zum Start wird eine einzelne Wegwerf-Messung durchgeführt, um einen Kaltstart zu vermeiden. Danach startet die eigentliche Messung über alle Kombinationen. Ist der gesamte Lauf beendet, wird der übergebene `done`-Callback aufgerufen, über den die Oberfläche informiert wird.

```
public void runAll(long sessionA, long sessionB, int numberConfig, Runnable done) {
```

```
    new Thread(() -> {
        String[] modelle   = {"1b", "2b", "qwen"};
        String[] szenarien = {"report", "artefakt", "vergleich"};

        // Aufwaermphase: eine Wegwerf-Inferenz gegen Kaltstart-Effekte
        try {
            LlmEngine warm = new LlmEngine(LlmEngine.pathFor(modelle[0]));
            warm.load(context);
            warm.question("Test");
            warm.close();
        } catch (Exception ignored) {}

        for (String modell : modelle) {
            for (String szenario : szenarien) {
                runConfig(modell, szenario, sessionA, sessionB, numberConfig);
            }
        }
        if (done != null) done.run();    // Benchmark fertig -> UI benachrichtigen
    }).start();
}
```

Listing 5.12: Ablauf des Benchmarks mit Aufwaermphase

Messung je Konfiguration. Für jede Kombination wird `runConfig` einmal aufgerufen. Dabei wird das Modell geladen und die Ladezeit gemessen, bevor dieselbe Anfrage n = 5 mal wiederholt wird (Listing 5.13). `questionStreaming` ist für die asynchrone Verarbeitung zuständig. Diese wird mithilfe eines `CountDownLatch` synchronisiert, der das Warten bei jeder Wiederholung übernimmt. Die Schleife wartet bei `await`, bis der `onDone`-Callback per `countDown` das Ende der Inferenz meldet. Erst danach werden die Speicher- und Akkuwerte ausgelesen und als eine Zeile gespeichert. Bei einem Fehler wird dies in der CSV-Datei gekennzeichnet.

```
for (int i = 0; i < nc; i++) {
    long[]   ttftHolder  = {0};
    long[]   totalHolder = {0};
    String[] textHolder  = {""};
    CountDownLatch latch = new CountDownLatch(1);

    engine.questionStreaming(prompt, new LlmEngine.StreamCallback() {
        @Override public void onChunk(String partial) { /* im Benchmark egal */ }
        @Override public void onDone(String fullText, long ttftMs, long totalMs) {
            ttftHolder[0]  = ttftMs;
            totalHolder[0] = totalMs;
            textHolder[0]  = fullText;
            latch.countDown();           // Riegel oeffnen
        }
    });

    latch.await();                       // warten, bis onDone kam

    long inferMs = totalHolder[0];
    int outputTokens = engine.tokenCount(textHolder[0]);
    double tokensPerSec = outputTokens / (inferMs / 1000.0);

    long pssMb   = Metrics.pssMb();
    long battUAh = Metrics.batteryUAh(context);
    int  battPct = Metrics.batteryPct(context);
```

```
        logger.logRow(modell, szenario, promptTokens, outputTokens,
                      loadMs, inferMs, ttftHolder[0], tokensPerSec,
                      pssMb, battUAh, battPct, "ok");
}
```

Listing 5.13: Wiederholte Messung mit Synchronisation ueber CountDownLatch

Erfassung der Ressourcen. Die Klasse `Metrics` ist eine Hilfsklasse, die die Speicher- und Akkuwerte liefert (Listing 5.14). Der Speicherverbrauch wird als PSS über `Debug.MemoryInfo` ausgelesen und von Kilobyte in Megabyte umgerechnet. Über den `BatteryManager` wird der aktuelle Ladezustand des Akkus in Prozent und Mikroamperestunden (µAh) ermittelt. Die Klasse `Metrics` bildet eine Hilfsklasse dar, von der aus die Speicher- und Akkuwerte geliefert werden (Listing 5.14). Der Speicherverbrauch wird als PSS über `Debug.MemoryInfo` ausgelesen und von Kilobyte in Megabyte umgerechnet. Über den `BatteryManager` wird der aktuelle Ladezustand des Akkus in Prozent und Mikroamperstunden (uAh) ermittelt.

```
public static long pssMb() {
    Debug.MemoryInfo mi = new Debug.MemoryInfo();
    Debug.getMemoryInfo(mi);
    return mi.getTotalPss() / 1024;    // KB -> MB
}

public static long batteryUAh(Context context) {
    BatteryManager bm = (BatteryManager) context.getSystemService(Context.BATTERY_SERV
    return bm.getLongProperty(BatteryManager.BATTERY_PROPERTY_CHARGE_COUNTER); // uAh
}

public static int batteryPct(Context context) {
    BatteryManager bm = (BatteryManager) context.getSystemService(Context.BATTERY_SERV
    return bm.getIntProperty(BatteryManager.BATTERY_PROPERTY_CAPACITY);
// Prozent
}
```

Listing 5.14: Auslesen von Speicher und Akku

Protokollierung. Die Klasse `BenchmarkLogger` schreibt die Ergebnisse in die Datei `benchmark.csv` (Listing 5.15). Ist die Datei schon vorhanden, wird die bestehende gelöscht und eine neue erstellt. Sie enthält eine Kopfzeile mit allen Metriken und darunter die einzelnen Zeilen, die jeweils eine Messung darstellen. Die so entstehende CSV bildet die Datengrundlage für die Evaluation (Kapitel 6). (Kapitel 6).

```
public BenchmarkLogger(Context context) {
    File dir = new File(context.getExternalFilesDir(
            Environment.DIRECTORY_DOCUMENTS), "UnicornART");
    if (!dir.exists()) dir.mkdirs();
    file = new File(dir, "benchmark.csv");

    if (file.exists()) file.delete();    // alte Datei loeschen -> frische CSV
    writeRow("timestamp,modell,szenario,promptTokens,outputTokens,loadMs,"
           + "inferMs,ttftMs,tokensPerSec,pssMb,batteryUAh,batteryPct,status");
}
```

Listing 5.15: Anlegen der CSV-Datei mit Kopfzeile

6 Evaluation

Dieses Kapitel präsentiert die Durchführung (Abschnitt 4.4.2) sowie die Ergebnisse der Metriken (Abschnitt 4.4.1), um zu beurteilen, inwiefern sich die lokalen Modelle für die Verarbeitung von EEG-Daten eignen.

6.1 Durchführung der Messungen

Wie schon in Abschnitt 4.4 beschrieben, wird die Eingabe für alle Modelle identisch gehalten. Insgesamt werden so alle neun Kombinationen aus drei Modellen und drei Szenarien auf dem Samsung Galaxy S23 vermessen und anschließend in einer neu erstellten CSV-Datei protokolliert. Dies geschieht je fünfmal, wobei die erste Messung jeder Kombination verworfen wird. Vor der ersten Messung wird ein Aufwärmlauf durchgeführt.

Als Eingabe für das Modell werden zwei bestimmte Sessions verwendet. Beide sind beim Tragen des Unicorn-Hybrid-Headsets entstanden und haben eine Laufzeit von knapp über 120 Sekunden.

6.2 Latenz

Im folgenden werden die Ergebnisse der Latenzmessungen vorgestellt.

Tabelle 6.1: Latenzmetriken je Modell und Szenario (Mittel aus vier Wiederholungen ± Standardabweichung).

Modell	Szenario	Ladezeit (s)	TTFT (s)	Gesamtdauer (s)	Tok/s
Gemma 3 1B	Report	0,45	1,94 ± 0,07	6,70 ± 0,14	21,65 ± 0,44
	Artefakt	0,34	1,95 ± 0,04	11,44 ± 0,04	25,34 ± 0,09
	Vergleich	0,29	5,86 ± 0,49	9,89 ± 0,46	11,85 ± 0,54
Qwen 2.5 1.5B	Report	0,73	5,36 ± 0,10	34,43 ± 0,24	14,06 ± 0,10
	Artefakt	0,27	5,31 ± 0,13	26,70 ± 0,12	13,37 ± 0,06
	Vergleich	0,27	13,99 ± 0,55	42,00 ± 1,94	11,31 ± 0,53
Gemma 2 2B	Report	0,87	6,09 ± 0,49	14,25 ± 0,49	5,06 ± 0,17
	Artefakt	0,47	6,75 ± 0,38	18,57 ± 0,58	4,74 ± 0,15
	Vergleich	0,52	16,60 ± 0,23	27,19 ± 0,13	2,87 ± 0,01

Tabelle 6.1 zeigt, dass der **Durchsatz (Tok/s)** bei dem Modell Gemma 3 1B am höchsten ist (bis zu 25,34 Tok/s), während das Modell Gemma 2 2B den geringsten Durchsatz aufweist (minimal 2,87 Tok/s). Die **Ladezeit** ist bei Gemma 2 2B am höchsten; die beiden kleineren Modelle Gemma 3 1B und Qwen 2.5 1.5B laden schneller. Sie liegt durchweg unter einer Sekunde (zwischen 0,27 und 0,87,s). Die **Zeit bis zum ersten Token (TTFT)** ist bei Gemma 3 1B am geringsten, während sie bei Gemma 2 2B am höchsten ist. Mit Ausnahme von

Qwen 2.5 1.5B/Artefakt steigt die TTFT mit dem Szenarioniveau an und erreicht ihren Höchstwert bei Gemma 2 2B mit 16,60,s. Die **Gesamtdauer** reicht von etwa 6,70 bis 42,00,s und variiert stark zwischen den Modellen und Szenarien.

Da die **Standardabweichungen** überwiegend gering ausfallen, weisen sie auf stabile Messungen hin. Lediglich bei Qwen 2.5 1.5B/Vergleich ist die Standardabweichung mit 1,94,s (Gesamtdauer) deutlich höher als bei allen anderen Messungen.

6.3 Ressourcenverbrauch

Daraufolgend werden die Ergebnisse der Messungen zum Ressourcenverbrauch vorgestellt.

Tabelle 6.2: Ressourcenverbrauch je Modell (RAM als Mittel ± Standardabweichung).

Modell	Szenario	RAM / PSS (MB)	Akku (mAh)*
Gemma 3 1B	Report	1482 ± 43	~70
	Artefakt	1508 ± 10	
	Vergleich	1615 ± 35	
Qwen 2.5 1.5B	Report	1994 ± 2	~200
	Artefakt	1994 ± 1	
	Vergleich	2114 ± 34	
Gemma 2 2B	Report	3691 ± 129	~100
	Artefakt	3638 ± 168	
	Vergleich	3798 ± 128	

*Verbrauch über den gesamten Messblock (15 Läufe) des jeweiligen Modells.

Tabelle 6.2 zeigt, dass der **RAM (PSS)** eine klare Rangfolge zwischen den Modellen besitzt. Während Gemma 3 1B mit (1482 MB) den geringsten Wert aufweist, liegt Qwen in der Mitte (ca. 2000 MB) und Gemma 2 2B am höchsten (3798 MB). Während das 1B-Modell von Gemma mit (ca. 70 mAh) den geringsten **Akkuverbrauch** aufweist, liegt der Verbrauch von Qwen 2.5 1.5B (ca. 200 mAh) und Gemma 2 2B (ca. 100 mAh) deutlich höher. Auffällig ist, dass das Modell Qwen am meisten Akku verbraucht.

6.4 Stabilität

Alle 45 Messungen wurden mit dem Status „ok“ protokolliert. Das bedeutet, dass alle Läufe erfolgreich liefen. Es gab keine Abstürze, keine Fehler und keinen Speicherüberlauf (). Das größere Modell Gemma 2 2B lief ebenso stabil, obwohl es mit ca. 3742,MB den meisten RAM verbrauchte. Auffällig war lediglich eine leichte Schwankung, die bei Gemma 2 2B/Artefakt auftrat. Dort fiel eine größere Streuung auf (± 2,24,s), die vermutlich unter Dauerlast entstand.

6.5 Ausgabequalität

6.5.1 Bewertungskriterien

- **Korrektheit:** Stimmt die Aussage mit den Merkmalen überein?
- **Verständlichkeit:** Ist die Ausgabe für Laien lesbar?
- **Relevanz:** Geht sie auf das Wesentliche ein?
- **Halluzination:** Werden Dinge erfunden, die nicht in den Daten stehen?

6.5.2 Beispielausgaben je Szenario

6.6 Zusammenfassende Gegenüberstellung

7 Diskussion

7.1 Einordnung der Ergebnisse

7.2 Einschränkungen

7.3 Handlungsempfehlungen

8 Fazit und Ausblick

8.1 Fazit

8.2 Ausblick

Literatur

[1] Nisreen Said Amer und Samir Brahim Belhaouari. „EEG Signal Processing for Medical Diagnosis, Healthcare, and Monitoring: A Comprehensive Review“. In: *IEEE Access* 11 (2023), S. 143116–143142. DOI: 10.1109/ACCESS.2023.3341419.

[2] Android Headlines. *One UI 8.5 Update Reaches Galaxy S23 Series Globally*. 2026. URL: https://www.androidheadlines.com/2026/05/one-ui-8-5-update-reaches-galaxy-s23-series-globally-whats-new.html (besucht am 19. 07. 2026).

[3] Sarah Blum, Stefan Debener, Reiner Emkes, Nils Volkening, Sebastian Fudickar und Martin G. Bleichner. „EEG Recording and Online Signal Processing on Android: A Multiapp Framework for Brain-Computer Interfaces on Smartphone“. In: *BioMed Research International* 2017 (2017), S. 3072870. DOI: 10.1155/2017/3072870.

[4] g.tec medical engineering. *Unicorn Bluetooth Protocol*. URL: https://github.com/unicorn-bi/Unicorn-Suite-Hybrid-Black/blob/master/Unicorn%20Bluetooth%20Protocol/UnicornBluetoothProtocol.pdf (besucht am 02. 07. 2026).

[5] g.tec medical engineering. *Unicorn Hybrid Black – User Manual*. URL: https://github.com/unicorn-bi/Unicorn-Suite-Hybrid-Black-User-Manual/blob/main/UnicornHybridBlack.md (besucht am 02. 07. 2026).

[6] g.tec medical engineering. *Unicorn Hybrid Black BCI Platform*. URL: https://www.gtec.at/product/unicorn-hybrid-black-bci-platform/ (besucht am 02. 07. 2026).

[7] Gemma Team. *Gemma 2: Improving Open Language Models at a Practical Size*. Techn. Ber. Google DeepMind, 2024. URL: https://arxiv.org/abs/2408.00118.

[8] Gemma Team. *Gemma 3 Technical Report*. Techn. Ber. Google DeepMind, 2025. URL: https://arxiv.org/abs/2503.19786.

[9] Gemma Team. *Gemma 4 Technical Report*. Techn. Ber. Google DeepMind, 2026. URL: https://ai.google.dev/gemma/docs/core/model_card_4?hl=de.

[10] Google. *BatteryManager – Android API reference*. 2024. URL: https://developer.android.com/reference/android/os/BatteryManager (besucht am 12. 07. 2026).

[11] Google. *Gemini Nano*. 2024. URL: https://developer.android.com/ai/gemini-nano (besucht am 24. 06. 2026).

[12] Google. *LLM Inference Guide – MediaPipe*. 2024. URL: https://developers.google.com/edge/mediapipe/solutions/genai/llm_inference (besucht am 24. 06. 2026).

[13] Google. *Memory allocation among processes*. 2024. URL: https://developer.android.com/topic/performance/memory-management (besucht am 12. 07. 2026).

[14] Google. *OutOfMemoryError – Android API reference*. 2024. URL: https://developer.android.com/reference/java/lang/OutOfMemoryError (besucht am 20. 07. 2026).

[15] Google. *Overview of the ML Kit GenAI APIs*. 2025. URL: https://developers.google.com/ml-kit/genai (besucht am 09. 07. 2026).

[16] Qingzhe Guo, Tu Ouyang und An Wang. „Large Language Models on Mobile Devices: A Measurement Study of Single- and Multi-Instance Execution“. In: *Proceedings of the 2nd International Workshop on Edge and Mobile Foundation Models (EdgeFM '25)*. Hong Kong, China: ACM, 2025. DOI: 10.1145/3737902.3768359.

[17] IBM. *Was ist ein Kontextfenster?* 2024. URL: https://www.ibm.com/de-de/think/topics/context-window (besucht am 12. 07. 2026).

[18] Mainak Jas, Denis A. Engemann, Yousra Bekhti, Federico Raimondo und Alexandre Gramfort. „Autoreject: Automated artifact rejection for MEG and EEG data“. In: *NeuroImage* 159 (2017), S. 417–429. DOI: 10.1016/j.neuroimage.2017.06.030.

[19] JGraph Ltd. *diagrams.net (draw.io)*. URL: https://www.drawio.com/ (besucht am 04. 07. 2026).

[20] Dah-Jing Jwo, Wei-Yeh Chang und I-Hua Wu. „Windowing Techniques, the Welch Method for Improvement of Power Spectrum Estimation“. In: *Computers, Materials & Continua* 67.3 (2021). DOI: 10.32604/cmc.2021.014752.

[21] Thomas Kuttner. *Praxiswissen Schwingungsmesstechnik*. 1. Aufl. Wiesbaden: Springer Vieweg, 2015. ISBN: 978-3-658-04637-8. DOI: 10.1007/978-3-658-04638-5.

[22] LiteRT Community. *Qwen2.5-1.5B-Instruct (LiteRT)*. 2025. URL: https://huggingface.co/litert-community/Qwen2.5-1.5B-Instruct (besucht am 16. 07. 2026).

[23] Qwen Team. *Qwen2.5 Technical Report*. Techn. Ber. Alibaba Group, 2024. URL: https://arxiv.org/abs/2412.15115.

[24] Qwen Team. *Qwen3 Technical Report*. Techn. Ber. Alibaba Group, 2025. URL: https://arxiv.org/abs/2505.09388.

[25] Samsung Electronics. *Galaxy S23 – Technische Daten*. 2023. URL: https://www.samsungmobilepress.com/media-assets/galaxy-s23?tab=specs (besucht am 19. 07. 2026).

[26] Steven W. Smith. *The Scientist and Engineer's Guide to Digital Signal Processing*. San Diego, CA: California Technical Publishing, 1997. ISBN: 978-0966017632. URL: https://www.dspguide.com/.

[27] Ashish Vaswani, Noam Shazeer, Niki Parmar u. a. „Attention Is All You Need“. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2017.

[28] Piotr Wendykier. *JTransforms*. URL: https://github.com/wendykierp/jtransforms (besucht am 02. 07. 2026).

[29] Yue Zheng, Yuhao Chen, Bin Qian, Xiufang Shi, Yuanchao Shu und Jiming Chen. „A Review on Edge Large Language Models: Design, Execution, and Applications“. In: *ACM Computing Surveys* 57.8 (2025). DOI: 10.1145/3719664.

Abbildungsverzeichnis

2.1 Überführung eines Signals vom Zeit- in den Frequenzbereich mittels FFT [19]. 4

3.1 Komponenten des verwendeten EEG-Geräts g.tec Unicorn Hybrid Black (eigene Beschriftung, erstellt mit draw.io) [6, 19]. 11

4.1 Aufteilung einer Session in vier gleich lange Zeitabschnitte [19]. 14
4.2 Zerlegung eines Segments in überlappende, mit einer Hann-Funktion gewichtete Analysefenster (50 % Überlappung) für die Welch-Methode [19]. 15

5.1 Architektur und Datenfluss der Anwendung [19] 20

Tabellenverzeichnis

2.1 Die fünf EEG-Frequenzbänder und ihre assoziierten Zustände [1] 3

3.1 Aufbau eines Samples des Unicorn Hybrid Black [4] 11

4.1 Übersicht der erfassten Metriken. 17

6.1 Latenzmetriken je Modell und Szenario (Mittel aus vier Wiederholungen $\pm$ Standardabweichung). 31
6.2 Ressourcenverbrauch je Modell (RAM als Mittel $\pm$ Standardabweichung). 32

Listings

5.1 Vorberechnung von Hann-Fenster und FFT-Objekt im Konstruktor 21
5.2 Vorabprüfung der Session 22
5.3 Zerlegung der Session in Segmente 22
5.4 Band-Power je Kanal nach dem Welch-Verfahren 23
5.5 Berechnung des Leistungsspektrums eines Fensters 23
5.6 Zuordnung der Frequenz-Bins zu den Bändern 24
5.7 Zentrale Definition der Frequenzbänder mit ihren Grenzen 25
5.8 Modellpfade und Auswahl des Modells 25
5.9 Laden und Konfigurieren des Modells 26
5.10 Streamende Ausgabe mit Messung der Latenz 26

5.11 Aufbau der Prompts je Szenario (finale, eingefrorene Fassung) 27
5.12 Ablauf des Benchmarks mit Aufwaermphase 28
5.13 Wiederholte Messung mit Synchronisation ueber CountDownLatch 29
5.14 Auslesen von Speicher und Akku 30
5.15 Anlegen der CSV-Datei mit Kopfzeile 30

Abkürzungsverzeichnis

Anhang

A Erster Abschnitt des Anhangs

In den Anhang gehören „Hintergrundinformationen“, also weiterführende Information, ausführliche Listings, Graphen, Diagramme oder Tabellen, die den Haupttext mit detaillierten Informationen ergänzen.

Dies hier ist ein Blindtext zum Testen von Textausgaben. Wer diesen Text liest, ist selbst schuld. Der Text gibt lediglich den Grauwert der Schrift an. Ist das wirklich so? Ist es gleichgültig, ob ich schreibe: „Dies ist ein Blindtext“ oder „Huardest gefburn“? Kjift – mitnichten! Ein Blindtext bietet mir wichtige Informationen. An ihm messe ich die Lesbarkeit einer Schrift, ihre Anmutung, wie harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln. Dies hier ist ein Blindtext zum Testen von Textausgaben. Wer diesen Text liest, ist selbst schuld. Der Text gibt lediglich den Grauwert der Schrift an. Ist das wirklich so? Ist es gleichgültig, ob ich schreibe: „Dies ist ein Blindtext“ oder „Huardest gefburn“? Kjift – mitnichten! Ein Blindtext bietet mir wichtige Informationen. An ihm messe ich die Lesbarkeit einer Schrift, ihre Anmutung, wie harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln. Dies hier ist ein Blindtext zum Testen von Textausgaben. Wer diesen Text liest, ist selbst schuld. Der Text gibt lediglich den Grauwert der Schrift an. Ist das wirklich so? Ist es gleichgültig, ob ich schreibe: „Dies ist ein Blindtext“ oder „Huardest gefburn“? Kjift – mitnichten! Ein Blindtext bietet mir wichtige Informationen. An ihm messe ich die Lesbarkeit einer Schrift, ihre Anmutung, wie harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln.

Kolophon

Dieses Dokument wurde mit der LaTeX-Vorlage für Abschlussarbeiten an der htw saar im Bereich Informatik/Mechatronik-Sensortechnik erstellt (Version 2.25, August 2024). Die Vorlage wurde von Yves Hary und André Miede entwickelt (mit freundlicher Unterstützung von Thomas Kretschmer, Helmut G. Folz und Martina Lehser). Daten: (F)10.95 – (B)426.79135pt – (H)688.5567pt